

```

void function(int n) {
    if( n < 2 ) return; // constant time
    else    counter = 0;
    // this loop executes 8 times with n value half in every call
    for i = 1 to 8 do
        function( $\frac{n}{2}$ );
    // this loop executes  $n^3$  times with constant time loop
    for i = 1 to  $n^3$  do
        counter = counter + 1;
}

```

$T(n)$ can be defined as follows:

$$\begin{aligned}
 T(n) &= 1 \text{ if } n < 2, \\
 &= 8T\left(\frac{n}{2}\right) + n^3 + 1 \text{ otherwise.}
 \end{aligned}$$

Using the master theorem gives: $T(n) = \Theta(n^{\log_2 8} \log n) = \Theta(n^3 \log n)$.

Problem-45 Find the complexity of the below pseudocode:

```

temp = 1
repeat
    for i = 1 to n
        temp = temp + 1;
    n =  $\frac{n}{2}$ ;
until n <= 1

```

Solution: Consider the comments in the pseudocode below:

```

temp = 1 // const time
repeat // this loops executes n times
    for i = 1 to n
        temp = temp + 1;
    // recursive call with  $\frac{n}{2}$  value
    n =  $\frac{n}{2}$ ;
until n <= 1

```

The recurrence for this function is $T(n) = T(n/2) + n$. Using master theorem, we get $T(n) = O(n)$.

Problem-46 Running time of the following program?

```
function(int n) {  
    for(int i = 1 ; i <= n ; i + + )  
        for(int j = 1 ; j <= n ; j * = 2 )  
            printf( " * " );  
}
```

Solution: Consider the comments in the below function:

```
function(int n) {  
    for(int i = 1 ; i <= n ; i + + ) // this loops executes n times  
        // this loops executes logn times from our logarithms guideline  
        for(int j = 1 ; j <= n ; j * = 2 )  
            printf( " * " );  
}
```

Complexity of above program is: $O(n \log n)$.

Problem-47 Running time of the following program?

```
function(int n) {  
    for(int i = 1 ; i <= n/3 ; i + + )  
        for(int j = 1 ; j <= n ; j += 4 )  
            printf( " * " );  
}
```

Solution: Consider the comments in the below function:

```
function(int n) { // this loops executes n/3 times  
    for(int i = 1 ; i <= n/3 ; i + + )  
        // this loops executes n/4 times  
        for(int j = 1 ; j <= n ; j += 4 )  
            printf( " * " );  
}
```

The time complexity of this program is: $O(n^2)$.

Problem-48 Find the complexity of the below function:

```

void function(int n) {
    if(n <= 1) return;
    if(n > 1) {
        printf (" * ");
        function(  $\frac{n}{2}$  );
        function(  $\frac{n}{2}$  );
    }
}

```

Solution: Consider the comments in the below function:

```

void function(int n) {
    if(n <= 1) return; //constant time
    if(n > 1) {
        //constant time
        printf (" * ");
        //recursion with n/2 value
        function( n/2 );
        //recursion with n/2 value
        function( n/2 );
    }
}

```

The recurrence for this function is: $T(n) = 2T\left(\frac{n}{2}\right) + 1$. Using master theorem, we get $T(n) = O(n)$.

Problem-49 Find the complexity of the below function:

```

function(int n) {
    int i=1;
    while (i < n) {
        int j=n;
        while(j > 0)
            j = j/2;
        i=2*i;
    } // i
}

```

Solution:

```
function(int n) {  
    int i=1;  
    while (i < n) {  
        int j=n;  
        while(j > 0)  
            j = j/2; //logn code  
        i=2*i; //logn times  
    }  
}
```

Time Complexity: $O(\log n * \log n) = O(\log^2 n)$.

Problem-50 $\sum_{i \leq k \leq n} O(n)$, where $O(n)$ stands for order n is:

- (A) $O(n)$
- (B) $O(n^2)$
- (C) $O(n^3)$
- (D) $O(3n^2)$
- (E) $O(1.5n^2)$

Solution: (B). $\sum_{i \leq k \leq n} O(n) = O(n) \sum_{i \leq k \leq n} 1 = O(n^2)$.

Problem-51 Which of the following three claims are correct?

- I $(n + k)^m = \Theta(n^m)$, where k and m are constants
 - II $2^{n+1} = O(2^n)$
 - III $2^{2n+1} = O(2^n)$
- (A) I and II
 - (B) I and III
 - (C) II and III
 - (D) I, II and III

Solution: (A). (I) $(n + k)^m = n^m + c_1 * n^{m-1} + \dots + k^m = \Theta(n^m)$ and (II) $2^{n+1} = 2 * 2^n = O(2^n)$

Problem-52 Consider the following functions:

$$f(n) = 2^n$$
$$g(n) = n!$$
$$h(n) = n^{\log n}$$

Which of the following statements about the asymptotic behavior of $f(n)$, $g(n)$, and $h(n)$ is true?

- (A) $f(n) = O(g(n)); g(n) = O(h(n))$
- (B) $f(n) = \Omega(g(n)); g(n) = O(h(n))$

- (C) $g(n) = O(f(n)); h(n) = O(f(n))$
 (D) $h(n) = O(f(n)); g(n) = \Omega(f(n))$

Solution: (D). According to the rate of growth: $h(n) < f(n) < g(n)$ ($g(n)$ is asymptotically greater than $f(n)$, and $f(n)$ is asymptotically greater than $h(n)$). We can easily see the above order by taking logarithms of the given 3 functions: $\log n \log n < n < \log(n!)$. Note that, $\log(n!) = O(n \log n)$.

Problem-53 Consider the following segment of C-code:

```
int j=1, n;
while (j <=n)
    j = j*2;
```

The number of comparisons made in the execution of the loop for any $n > 0$ is:

- (A) $\text{ceil}(\log_2^n) + 1$
 (B) n
 (C) $\text{ceil}(\log_2^n)$
 (D) $\text{floor}(\log_2^n) + 1$

Solution: (a). Let us assume that the loop executes k times. After k^{th} step the value of j is 2^k . Taking logarithms on both sides gives $k = \log_2^n$. Since we are doing one more comparison for exiting from the loop, the answer is $\text{ceil}(\log_2^n) + 1$.

Problem-54 Consider the following C code segment. Let $T(n)$ denote the number of times the for loop is executed by the program on input n . Which of the following is true?

```
int IsPrime(int n){
    for(int i=2;i<=sqrt(n);i++)
        if(n%i == 0){
            printf("Not Prime\n");
            return 0;
        }
    return 1;
}
```

- (A) $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(\sqrt{n})$
 (B) $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(1)$
 (C) $T(n) = O(n)$ and $T(n) = \Omega(\sqrt{n})$
 (D) None of the above

Solution: (B). Big O notation describes the tight upper bound and Big Omega notation describes the tight lower bound for an algorithm. The *for* loop in the question is run maximum $\sqrt{n}$ times and

minimum 1 time. Therefore, $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(1)$.

Problem-55 In the following C function, let $n \geq m$. How many recursive calls are made by this function?

```
int gcd(n,m){
    if (n%m ==0)
        return m;
    n = n%m;
    return gcd(m,n);
}
```

- (A) $\Theta(\log_2^n)$
- (B) $\Omega(n)$
- (C) $\Theta(\log_2 \log_2^n)$
- (D) $\Theta(n)$

Solution: No option is correct. Big O notation describes the tight upper bound and Big Omega notation describes the tight lower bound for an algorithm. For $m = 2$ and for all $n = 2^i$, the running time is $O(1)$ which contradicts every option.

Problem-56 Suppose $T(n) = 2T(n/2) + n$, $T(0)=T(1)=1$. Which one of the following is false?

- (A) $T(n) = O(n^2)$
- (B) $T(n) = \Theta(n \log n)$
- (C) $T(n) = Q(n^2)$
- (D) $T(n) = O(n \log n)$

Solution: (C). Big O notation describes the tight upper bound and Big Omega notation describes the tight lower bound for an algorithm. Based on master theorem, we get $T(n) = \Theta(n \log n)$. This indicates that tight lower bound and tight upper bound are the same. That means, $O(n \log n)$ and $\Omega(n \log n)$ are correct for given recurrence. So option (C) is wrong.

Problem-57 Find the complexity of the below function:

```

function(int n) {
    for (int i = 0; i<n; i++)
        for(int j=i; j<i*i; j++)
            if (j %i == 0){
                for (int k = 0; k < j; k++)
                    printf(" * ");
            }
}

```

Solution:

```

function(int n) {
    for (int i = 0; i<n; i++)           // Executes n times
        for(int j=i; j<i*i; j++)       // Executes n*n times
            if (j %i == 0){
                for (int k = 0; k < j; k++) // Executes j times = (n*n) times
                    printf(" * ");
            }
}

```

Time Complexity: $O(n^5)$.

Problem-58 To calculate 9^n , give an algorithm and discuss its complexity.

Solution: Start with 1 and multiply by 9 until reaching 9^n .

Time Complexity: There are $n - 1$ multiplications and each takes constant time giving a $\Theta(n)$ algorithm.

Problem-59 For [Problem-58](#), can we improve the time complexity?

Solution: Refer to the [Divide and Conquer](#) chapter.

Problem-60 Find the time complexity of recurrence $T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + T(\frac{n}{8}) + n$.

Solution: Let us solve this problem by method of guessing. The total size on each level of the recurrence tree is less than n , so we guess that $f(n) = n$ will dominate. Assume for all $i < n$ that $c_1n \leq T(i) < c_2n$. Then,

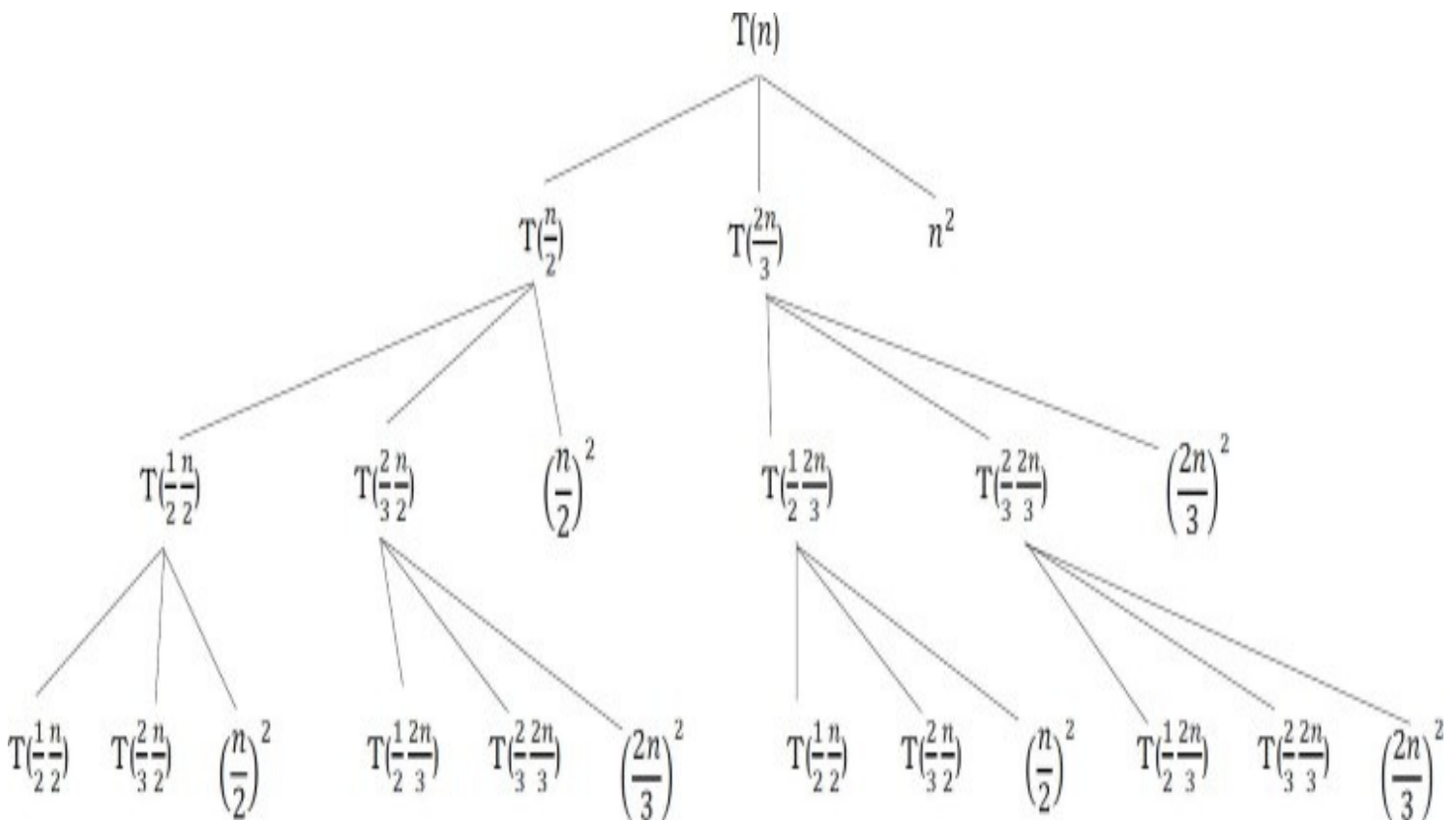
$$\begin{aligned}
c_1 \frac{n}{2} + c_1 \frac{n}{4} + c_1 \frac{n}{8} + kn &\leq T(n) \leq c_2 \frac{n}{2} + c_2 \frac{n}{4} + c_2 \frac{n}{8} + kn \\
c_1 n \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{k}{c_1} \right) &\leq T(n) \leq c_2 n \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{k}{c_2} \right) \\
c_1 n \left(\frac{7}{8} + \frac{k}{c_1} \right) &\leq T(n) \leq c_2 n \left(\frac{7}{8} + \frac{k}{c_2} \right)
\end{aligned}$$

If $c_1 \geq 8k$ and $c_2 \leq 8k$, then $c_1 n = T(n) = c_2 n$. So, $T(n) = \Theta(n)$. In general, if you have multiple recursive calls, the sum of the arguments to those calls is less than n (in this case $\frac{n}{2} + \frac{n}{4} + \frac{n}{8} < n$), and $f(n)$ is reasonably large, a good guess is $T(n) = \Theta(f(n))$.

Problem-61 Solve the following recurrence relation using the recursion tree method:

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{2n}{3}\right) + n^2.$$

Solution: How much work do we do in each level of the recursion tree?



$$\left(\frac{1}{4}n\right)^2 + \left(\frac{1}{3}n\right)^2 + \left(\frac{1}{3}\right)n^2 + \left(\frac{4}{9}\right)n^2 = \frac{625}{1296}n^2 = \left(\frac{25}{36}\right)^2 n^2$$

Similarly the amount of work at level k is at most $\left(\frac{25}{36}\right)^k n^2$.

Let $\alpha = \frac{25}{36}$, the total runtime is then:

$$\begin{aligned} T(n) &\leq \sum_{k=0}^{\infty} \alpha^k n^2 \\ &= \frac{1}{1-\alpha} n^2 \\ &= \frac{1}{1-\frac{25}{36}} n^2 \\ &= \frac{1}{\frac{11}{36}} n^2 \\ &= \frac{36}{11} n^2 \\ &= O(n^2) \end{aligned}$$

That is, the first level provides a constant fraction of the total runtime.

Problem-62 Rank the following functions by order of growth: $(n+1)!$, $n!$, 4^n , $n \times 3^n$, $3^n + n^2 + 20n$, $\left(\frac{3}{2}\right)^n$, $n^2 + 200$, $20n + 500$, $2^{\lg n}$, $n^{2/3}$, 1.

Solution:


```

function(int n) {
    int sum = 0;
    for (int i = 0; i < n; i++)           // Executes n times
        if (i > j)
            sum = sum + 1;               // Executes n times
        else {
            for (int k = 0; k < n; k++)    // Executes n times
                sum = sum - 1;
        }
    }
}

```

Time Complexity: $O(n^2)$.

Problem-64 Can we say $3^{n^{0.75}} = O(3^n)$??

Solution: Yes: because $3^{n^{0.75}} < 3^{n^1}$

Problem-65 Can we say $2^{3n} = O(2^n)$?

Solution: No: because $2^{3n} = (2^3)^n = 8^n$ not less than 2^n .

RECURSION AND BACKTRACKING

CHAPTER

2



2.1 Introduction

In this chapter, we will look at one of the important topics, “*recursion*”, which will be used in almost every chapter, and also its relative “*backtracking*”.

2.2 What is Recursion?

Any function which calls itself is called *recursive*. A recursive method solves a problem by calling a copy of itself to work on a smaller problem. This is called the recursion step. The recursion step can result in many more such recursive calls.

It is important to ensure that the recursion terminates. Each time the function calls itself with a slightly simpler version of the original problem. The sequence of smaller problems must eventually converge on the base case.

2.3 Why Recursion?

Recursion is a useful technique borrowed from mathematics. Recursive code is generally shorter and easier to write than iterative code. Generally, loops are turned into recursive functions when they are compiled or interpreted.

Recursion is most useful for tasks that can be defined in terms of similar subtasks. For example, sort, search, and traversal problems often have simple recursive solutions.

2.4 Format of a Recursive Function

A recursive function performs a task in part by calling itself to perform the subtasks. At some point, the function encounters a subtask that it can perform without calling itself. This case, where the function does not recur, is called the *base case*. The former, where the function calls itself to perform a subtask, is referred to as the *recursive case*. We can write all recursive functions using the format:

```
if(test for the base case)
    return some base case value
else if(test for another base case)
    return some other base case value
// the recursive case
else
    return (some work and then a recursive call)
```

As an example consider the factorial function: $n!$ is the product of all integers between n and 1. The definition of recursive factorial looks like:

$$\begin{aligned} n! &= 1, & \text{if } n &= 0 \\ n! &= n * (n - 1)! & \text{if } n > 0 \end{aligned}$$

This definition can easily be converted to recursive implementation. Here the problem is determining the value of $n!$, and the subproblem is determining the value of $(n - 1)!$. In the recursive case, when n is greater than 1, the function calls itself to determine the value of $(n - 1)!$ and multiplies that with n .

In the base case, when n is 0 or 1, the function simply returns 1. This looks like the following:

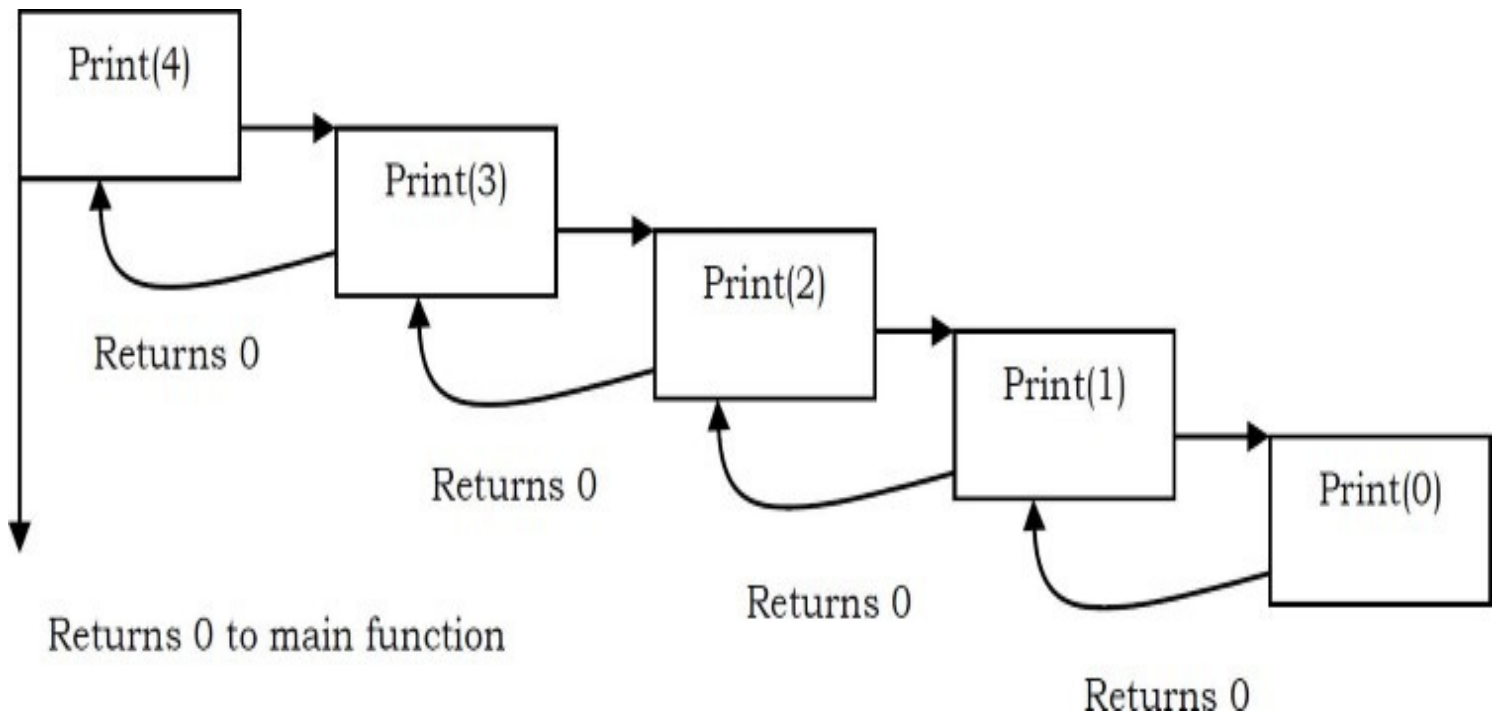
```
// calculates factorial of a positive integer
int Fact(int n) {
    if(n == 1) // base cases: fact of 0 or 1 is 1
        return 1;
    else if(n == 0)
        return 1;
    else // recursive case: multiply n by (n - 1) factorial
        return n*Fact(n-1);
}
```

2.5 Recursion and Memory (Visualization)

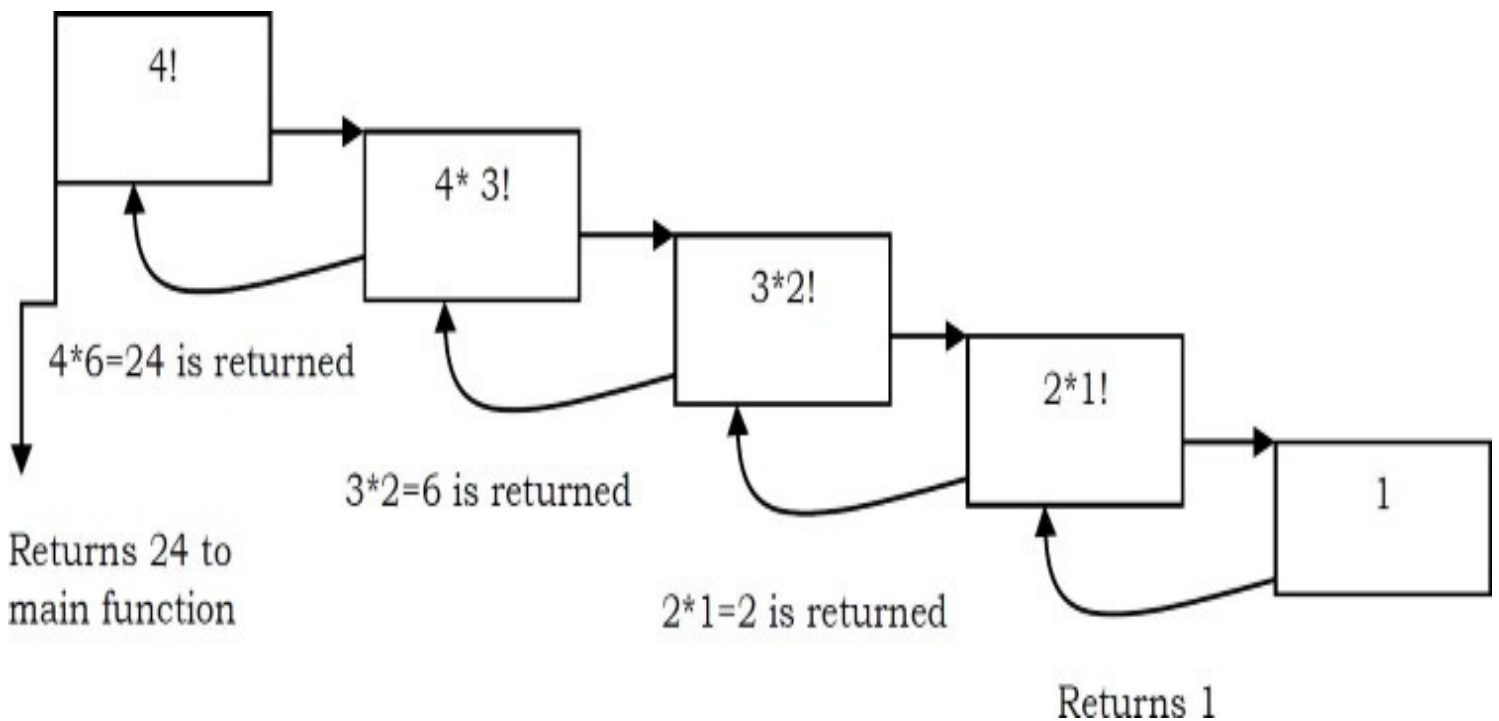
Each recursive call makes a new copy of that method (actually only the variables) in memory. Once a method ends (that is, returns some data), the copy of that returning method is removed from memory. The recursive solutions look simple but visualization and tracing takes time. For better understanding, let us consider the following example.

```
//print numbers 1 to n backward
int Print(int n) {
    if( n == 0) // this is the terminating base case
        return 0;
    else {
        printf ("%d",n);
        return Print(n-1); // recursive call to itself again
    }
}
```

For this example, if we call the print function with $n=4$, visually our memory assignments may look like:



Now, let us consider our factorial function. The visualization of factorial function with $n=4$ will look like:



2.6 Recursion versus Iteration

While discussing recursion, the basic question that comes to mind is: which way is better? – iteration or recursion? The answer to this question depends on what we are trying to do. A recursive approach mirrors the problem that we are trying to solve. A recursive approach makes it simpler to solve a problem that may not have the most obvious of answers. But, recursion adds

overhead for each recursive call (needs space on the stack frame).

Recursion

- Terminates when a base case is reached.
- Each recursive call requires extra space on the stack frame (memory).
- If we get infinite recursion, the program may run out of memory and result in stack overflow.
- Solutions to some problems are easier to formulate recursively.

Iteration

- Terminates when a condition is proven to be false.
- Each iteration does not require extra space.
- An infinite loop could loop forever since there is no extra memory being created.
- Iterative solutions to a problem may not always be as obvious as a recursive solution.

2.7 Notes on Recursion

- Recursive algorithms have two types of cases, recursive cases and base cases.
- Every recursive function case must terminate at a base case.
- Generally, iterative solutions are more efficient than recursive solutions [due to the overhead of function calls].
- A recursive algorithm can be implemented without recursive function calls using a stack, but it's usually more trouble than its worth. That means any problem that can be solved recursively can also be solved iteratively.
- For some problems, there are no obvious iterative algorithms.
- Some problems are best suited for recursive solutions while others are not.

2.8 Example Algorithms of Recursion

- Fibonacci Series, Factorial Finding
- Merge Sort, Quick Sort
- Binary Search
- Tree Traversals and many Tree Problems: InOrder, PreOrder PostOrder
- Graph Traversals: DFS [Depth First Search] and BFS [Breadth First Search]
- Dynamic Programming Examples
- Divide and Conquer Algorithms
- Towers of Hanoi

- Backtracking Algorithms [we will discuss in next section]

2.9 Recursion: Problems & Solutions

In this chapter we cover a few problems with recursion and we will discuss the rest in other chapters. By the time you complete reading the entire book, you will encounter many recursion problems.

Problem-1 Discuss Towers of Hanoi puzzle.

Solution: The Towers of Hanoi is a mathematical puzzle. It consists of three rods (or pegs or towers), and a number of disks of different sizes which can slide onto any rod. The puzzle starts with the disks on one rod in ascending order of size, the smallest at the top, thus making a conical shape. The objective of the puzzle is to move the entire stack to another rod, satisfying the following rules:

- Only one disk may be moved at a time.
- Each move consists of taking the upper disk from one of the rods and sliding it onto another rod, on top of the other disks that may already be present on that rod.
- No disk may be placed on top of a smaller disk.

Algorithm:

- Move the top $n - 1$ disks from *Source* to *Auxiliary* tower,
- Move the n^{th} disk from *Source* to *Destination* tower,
- Move the $n - 1$ disks from *Auxiliary* tower to *Destination* tower.
- Transferring the top $n - 1$ disks from *Source* to *Auxiliary* tower can again be thought of as a fresh problem and can be solved in the same manner. Once we solve *Towers of Hanoi* with three disks, we can solve it with any number of disks with the above algorithm.

```

void TowersOfHanoi(int n, char frompeg, char topeg, char auxpeg) {
    /* If only 1 disk, make the move and return */
    if(n==1) {
        printf("Move disk 1 from peg %c to peg %c",frompeg, topeg);
        return;
    }
    /* Move top n-1 disks from A to B, using C as auxiliary */
    TowersOfHanoi(n-1, frompeg, auxpeg, topeg);

    /* Move remaining disks from A to C */
    printf("\nMove disk %d from peg %c to peg %c", n, frompeg, topeg);

    /* Move n-1 disks from B to C using A as auxiliary */
    TowersOfHanoi(n-1, auxpeg, topeg, frompeg);
}

```

Problem-2 Given an array, check whether the array is in sorted order with recursion.

Solution:

```

int isArrayInSortedOrder(int A[],int n){
    if(n == 1)
        return 1;
    return (A[n-1] < A[n-2])?0:isArrayInSortedOrder(A,n-1);
}

```

Time Complexity: $O(n)$. Space Complexity: $O(n)$ for recursive stack space.

2.10 What is Backtracking?

Backtracking is an improvement of the brute force approach. It systematically searches for a solution to a problem among all available options. In backtracking, we start with one possible option out of many available options and try to solve the problem if we are able to solve the problem with the selected move then we will print the solution else we will backtrack and select some other option and try to solve it. If none of the options work out we will claim that there is no solution for the problem.

Backtracking is a form of recursion. The usual scenario is that you are faced with a number of options, and you must choose one of these. After you make your choice you will get a new set of

options; just what set of options you get depends on what choice you made. This procedure is repeated over and over until you reach a final state. If you made a good sequence of choices, your final state is a goal state; if you didn't, it isn't.

Backtracking can be thought of as a selective tree/graph traversal method. The tree is a way of representing some initial starting position (the root node) and a final goal state (one of the leaves). Backtracking allows us to deal with situations in which a raw brute-force approach would explode into an impossible number of options to consider. Backtracking is a sort of refined brute force. At each node, we eliminate choices that are obviously not possible and proceed to recursively check only those that have potential.

What's interesting about backtracking is that we back up only as far as needed to reach a previous decision point with an as-yet-unexplored alternative. In general, that will be at the most recent decision point. Eventually, more and more of these decision points will have been fully explored, and we will have to backtrack further and further. If we backtrack all the way to our initial state and have explored all alternatives from there, we can conclude the particular problem is unsolvable. In such a case, we will have done all the work of the exhaustive recursion and known that there is no viable solution possible.

- Sometimes the best algorithm for a problem is to try all possibilities.
- This is always slow, but there are standard tools that can be used to help.
- Tools: algorithms for generating basic objects, such as binary strings [2^n possibilities for n -bit string], permutations [$n!$], combinations [$n!/r!(n - r)!$], general strings [k -ary strings of length n has k^n possibilities], etc...
- Backtracking speeds the exhaustive search by pruning.

2.11 Example Algorithms of Backtracking

- Binary Strings: generating all binary strings
- Generating k – ary Strings
- N-Queens Problem
- The Knapsack Problem
- Generalized Strings
- Hamiltonian Cycles [refer to [Graphs](#) chapter]
- Graph Coloring Problem

2.12 Backtracking: Problems & Solutions

Problem-3 Generate all the strings of n bits. Assume $A[0..n - 1]$ is an array of size n .

Solution:

```

void Binary(int n) {
    if(n < 1)
        printf("%s", A);           //Assume array A is a global variable
    else {
        A[n-1] = 0;
        Binary(n - 1);
        A[n-1] = 1;
        Binary(n - 1);
    }
}

```

Let $T(n)$ be the running time of $binary(n)$. Assume function $printf$ takes time $O(1)$.

$$T(n) = \begin{cases} c, & \text{if } n < 0 \\ 2T(n - 1) + d, & \text{otherwise} \end{cases}$$

Using Subtraction and Conquer Master theorem we get: $T(n) = O(2^n)$. This means the algorithm for generating bit-strings is optimal.

Problem-4 Generate all the strings of length n drawn from $0 \dots k - 1$.

Solution: Let us assume we keep current k -ary string in an array $A[0.. n - 1]$. Call function $k\text{-string}(n, k)$:

```

void k-string(int n, int k) {
    //process all k-ary strings of length m
    if(n < 1)
        printf("%s", A);           //Assume array A is a global variable
    else {
        for (int j = 0 ; j < k ; j++) {
            A[n-1] = j;
            k-string(n- 1, k);
        }
    }
}

```

Let $T(n)$ be the running time of $k\text{-string}(n)$. Then,

$$T(n) = \begin{cases} c, & \text{if } n < 0 \\ kT(n-1) + d, & \text{otherwise} \end{cases}$$

Using Subtraction and Conquer Master theorem we get: $T(n) = O(k^n)$.

Note: For more problems, refer to [String Algorithms](#) chapter.

Problem-5 Finding the length of connected cells of 1s (regions) in an matrix of Os and 1s: Given a matrix, each of which may be 1 or 0. The filled cells that are connected form a region. Two cells are said to be connected if they are adjacent to each other horizontally, vertically or diagonally. There may be several regions in the matrix. How do you find the largest region (in terms of number of cells) in the matrix?

Sample Input:	11000 01100 00101 10001 01011	Sample Output:	5
---------------	---	----------------	---

Solution: The simplest idea is: for each location traverse in all 8 directions and in each of those directions keep track of maximum region found.

```

int getval(int (*A)[5],int i,int j,int L, int H){
    if (i< 0 || i >= L || j< 0 || j >= H)
        return 0;
    else
        return A[i][j];
}

void findMaxBlock(int (*A)[5], int r, int c,int L,int H,int size, bool **cntarr,int &maxsize){
    if ( r >= L || c >= H)
        return;
    cntarr[r][c]=true;
    size++;
    if (size > maxsize)
        maxsize = size;
    //search in eight directions
    int direction[][2]={{-1,0},{-1,-1},{0,-1},{1,-1},{1,0},{1,1},{0,1},{-1,1}};
    for(int i=0; i<8; i++) {
        int newi =r+direction[i][0];
        int newj=c+direction[i][1];
        int val=getval (A,newi,newj,L,H);
        if (val>0 && (cntarr[newi][newj]==false)){
            findMaxBlock(A,newi,newj,L,H,size,cntarr,maxsize);
        }
    }
    cntarr[r][c]=false;
}

int getMaxOnes(int (*A)[5], int rmax, int colmax){
    int maxsize=0;
    int size=0;
    bool **cntarr=create2darr(rmax,colmax);
    for(int i=0; i< rmax; i++){
        for(int j=0; j< colmax; j++){
            if (A[i][j] == 1){
                findMaxBlock(A,i,j,rmax,colmax, 0,cntarr,maxsize);
            }
        }
    }
    return maxsize;
}

```

Sample Call:

```
int zarr[][5]={1,1,0,0,0},{0,1,1,0,1},{0,0,0,1,1},{1,0,0,1,1},{0,1,0,1,1};  
cout << "Number of maximum 1s are " << getMaxOnes(zarr,5,5) << endl;
```

Problem-6 Solve the recurrence $T(n) = 2T(n - 1) + 2^n$.

Solution: At each level of the recurrence tree, the number of problems is double from the previous level, while the amount of work being done in each problem is half from the previous level. Formally, the i^{th} level has 2^i problems, each requiring 2^{n-i} work. Thus the i^{th} level requires exactly 2^n work. The depth of this tree is n , because at the i^{th} level, the originating call will be $T(n - i)$. Thus the total complexity for $T(n)$ is $T(n2^n)$.

LINKED LISTS

CHAPTER

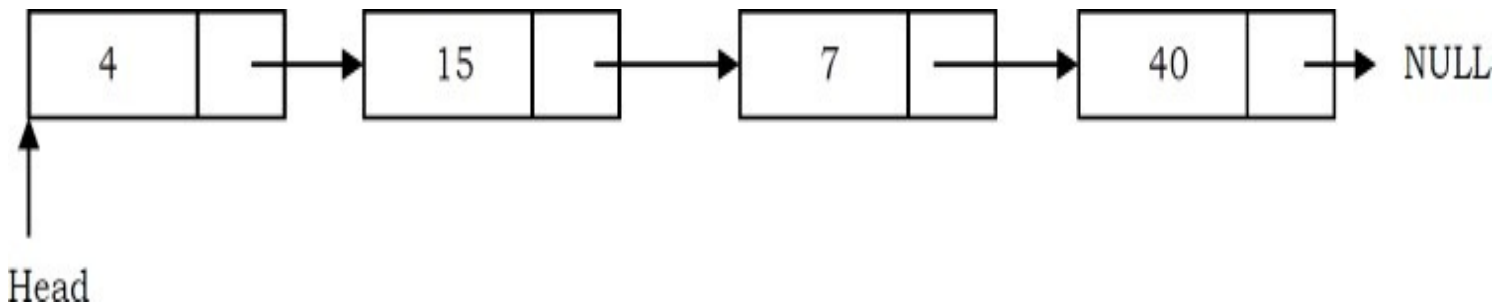
3



3.1 What is a Linked List?

A linked list is a data structure used for storing collections of data. A linked list has the following properties.

- Successive elements are connected by pointers
- The last element points to NULL
- Can grow or shrink in size during execution of a program
- Can be made just as long as required (until systems memory exhausts)
- Does not waste memory space (but takes some extra memory for pointers). It allocates memory as list grows.



3.2 Linked Lists ADT

The following operations make linked lists an ADT:

Main Linked Lists Operations

- Insert: inserts an element into the list
- Delete: removes and returns the specified position element from the list

Auxiliary Linked Lists Operations

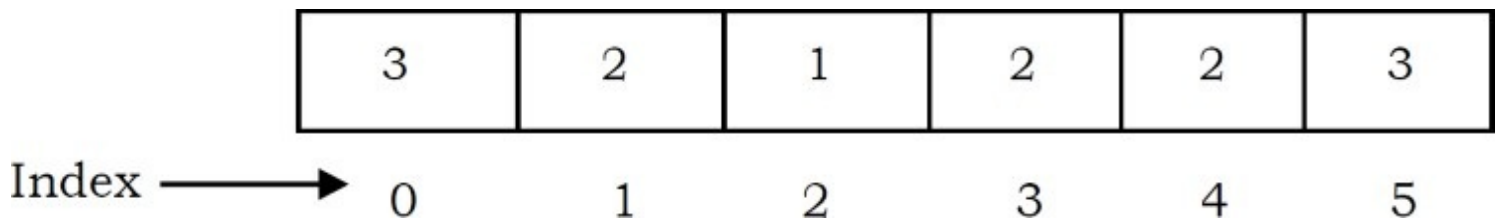
- Delete List: removes all elements of the list (disposes the list)
- Count: returns the number of elements in the list
- Find n^{th} node from the end of the list

3.3 Why Linked Lists?

There are many other data structures that do the same thing as linked lists. Before discussing linked lists it is important to understand the difference between linked lists and arrays. Both linked lists and arrays are used to store collections of data, and since both are used for the same purpose, we need to differentiate their usage. That means in which cases *arrays* are suitable and in which cases *linked lists* are suitable.

3.4 Arrays Overview

One memory block is allocated for the entire array to hold the elements of the array. The array elements can be accessed in constant time by using the index of the particular element as the subscript.



Why Constant Time for Accessing Array Elements?

To access an array element, the address of an element is computed as an offset from the base address of the array and one multiplication is needed to compute what is supposed to be added to the base address to get the memory address of the element. First the size of an element of that data type is calculated and then it is multiplied with the index of the element to get the value to be added to the base address.

This process takes one multiplication and one addition. Since these two operations take constant time, we can say the array access can be performed in constant time.

Advantages of Arrays

- Simple and easy to use
- Faster access to the elements (constant access)

Disadvantages of Arrays

- Preallocates all needed memory up front and wastes memory space for indices in the array that are empty.
- **Fixed size:** The size of the array is static (specify the array size before using it).
- **One block allocation:** To allocate the array itself at the beginning, sometimes it may not be possible to get the memory for the complete array (if the array size is big).
- **Complex position-based insertion:** To insert an element at a given position, we may need to shift the existing elements. This will create a position for us to insert the new element at the desired position. If the position at which we want to add an element is at the beginning, then the shifting operation is more expensive.

Dynamic Arrays

Dynamic array (also called as *growable array*, *resizable array*, *dynamic table*, or *array list*) is a random access, variable-size list data structure that allows elements to be added or removed.

One simple way of implementing dynamic arrays is to initially start with some fixed size array. As soon as that array becomes full, create the new array double the size of the original array.

Similarly, reduce the array size to half if the elements in the array are less than half.

Note: We will see the implementation for *dynamic arrays* in the *Stacks*, *Queues* and *Hashing* chapters.

Advantages of Linked Lists

Linked lists have both advantages and disadvantages. The advantage of linked lists is that they can be *expanded* in constant time. To create an array, we must allocate memory for a certain number of elements. To add more elements to the array when full, we must create a new array and copy the old array into the new array. This can take a lot of time.

We can prevent this by allocating lots of space initially but then we might allocate more than we need and waste memory. With a linked list, we can start with space for just one allocated element and *add* on new elements easily without the need to do any copying and reallocating.

Issues with Linked Lists (Disadvantages)

There are a number of issues with linked lists. The main disadvantage of linked lists is *access time* to individual elements. Array is random-access, which means it takes $O(1)$ to access any element in the array. Linked lists take $O(n)$ for access to an element in the list in the worst case. Another advantage of arrays in access time is *spacial locality* in memory. Arrays are defined as contiguous blocks of memory, and so any array element will be physically near its neighbors. This greatly benefits from modern CPU caching methods.

Although the dynamic allocation of storage is a great advantage, the *overhead* with storing and retrieving data can make a big difference. Sometimes linked lists are *hard to manipulate*. If the last item is deleted, the last but one must then have its pointer changed to hold a NULL reference. This requires that the list is traversed to find the last but one link, and its pointer set to a NULL reference.

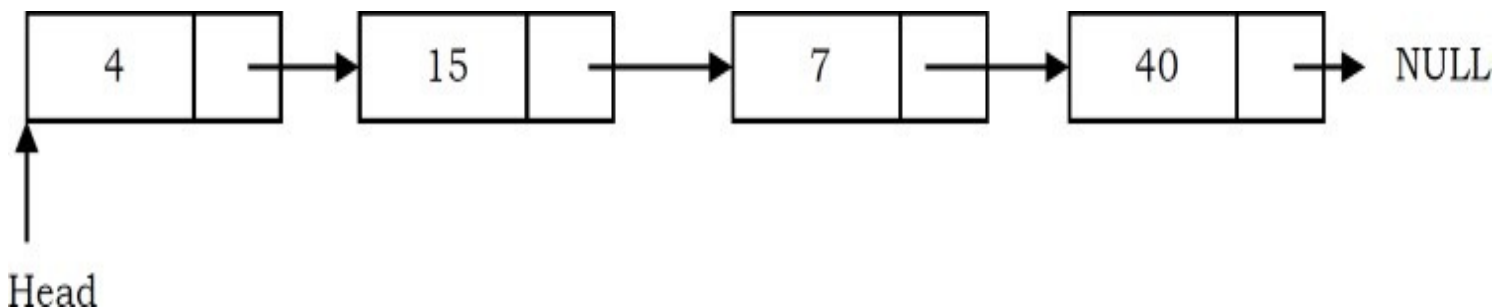
Finally, linked lists waste memory in terms of extra reference points.

3.5 Comparison of Linked Lists with Arrays & Dynamic Arrays

Parameter	Linked List	Array	Dynamic Array
Indexing	$O(n)$	$O(1)$	$O(1)$
Insertion/deletion at beginning	$O(1)$	$O(n)$, if array is not full (for shifting the elements)	$O(n)$
Insertion at ending	$O(n)$	$O(1)$, if array is not full	$O(1)$, if array is not full $O(n)$, if array is full
Deletion at ending	$O(n)$	$O(1)$	$O(n)$
Insertion in middle	$O(n)$	$O(n)$, if array is not full (for shifting the elements)	$O(n)$
Deletion in middle	$O(n)$	$O(n)$, if array is not full (for shifting the elements)	$O(n)$
Wasted space	$O(n)$ (for pointers)	0	$O(n)$

3.6 Singly Linked Lists

Generally “linked list” means a singly linked list. This list consists of a number of nodes in which each node has a *next* pointer to the following element. The link of the last node in the list is NULL, which indicates the end of the list.



Following is a type declaration for a linked list of integers:

```

struct ListNode {
    int data;
    struct ListNode *next;
};
  
```

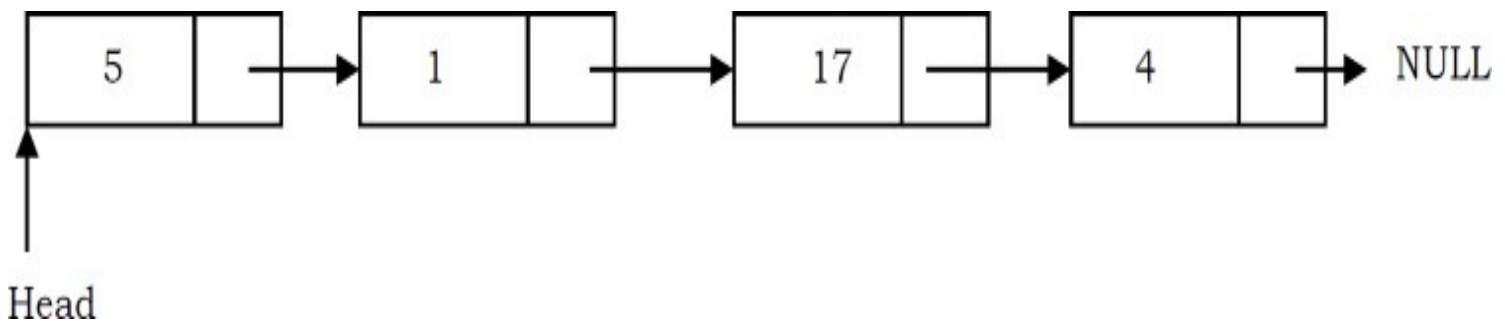
Basic Operations on a List

- Traversing the list
- Inserting an item in the list
- Deleting an item from the list

Traversing the Linked List

Let us assume that the *head* points to the first node of the list. To traverse the list we do the following

- Follow the pointers.
- Display the contents of the nodes (or count) as they are traversed.
- Stop when the next pointer points to NULL.



The `ListLength()` function takes a linked list as input and counts the number of nodes in the list. The function given below can be used for printing the list data with extra print function.

```

int ListLength(struct ListNode *head) {
    struct ListNode *current = head;
    int count = 0;

    while (current != NULL) {
        count++;
        current = current->next;
    }
    return count;
}
  
```

Time Complexity: $O(n)$, for scanning the list of size n .

Space Complexity: $O(1)$, for creating a temporary variable.

Singly Linked List Insertion

Insertion into a singly-linked list has three cases:

- Inserting a new node before the head (at the beginning)
- Inserting a new node after the tail (at the end of the list)
- Inserting a new node at the middle of the list (random location)

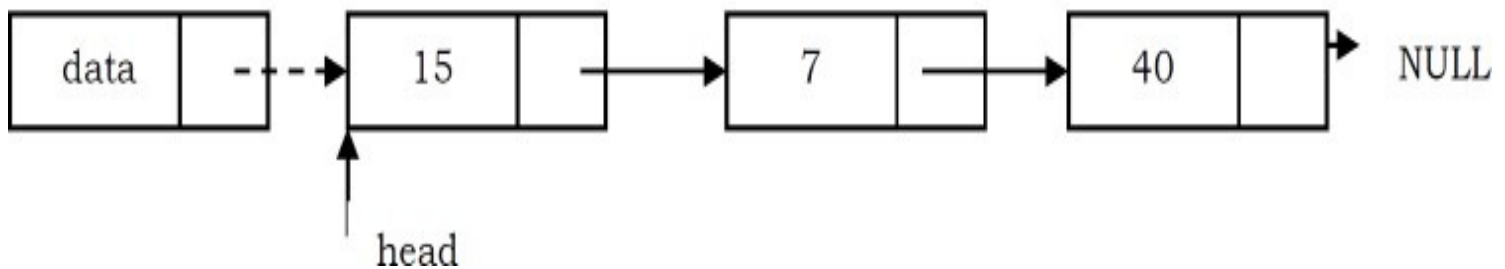
Note: To insert an element in the linked list at some position p , assume that after inserting the element the position of this new node is p .

Inserting a Node in Singly Linked List at the Beginning

In this case, a new node is inserted before the current head node. *Only one next pointer* needs to be modified (*new node's next pointer*) and it can be done in two steps:

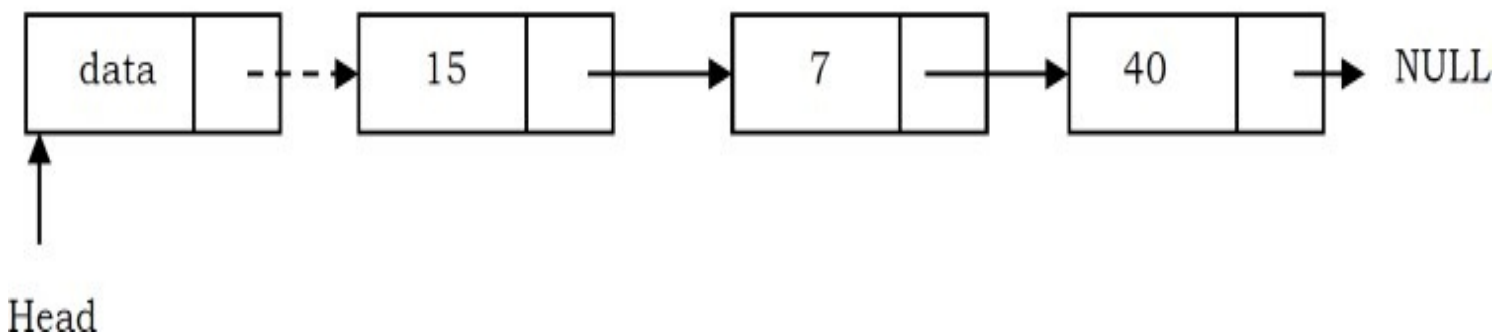
- Update the next pointer of new node, to point to the current head.

New node



- Update head pointer to point to the new node.

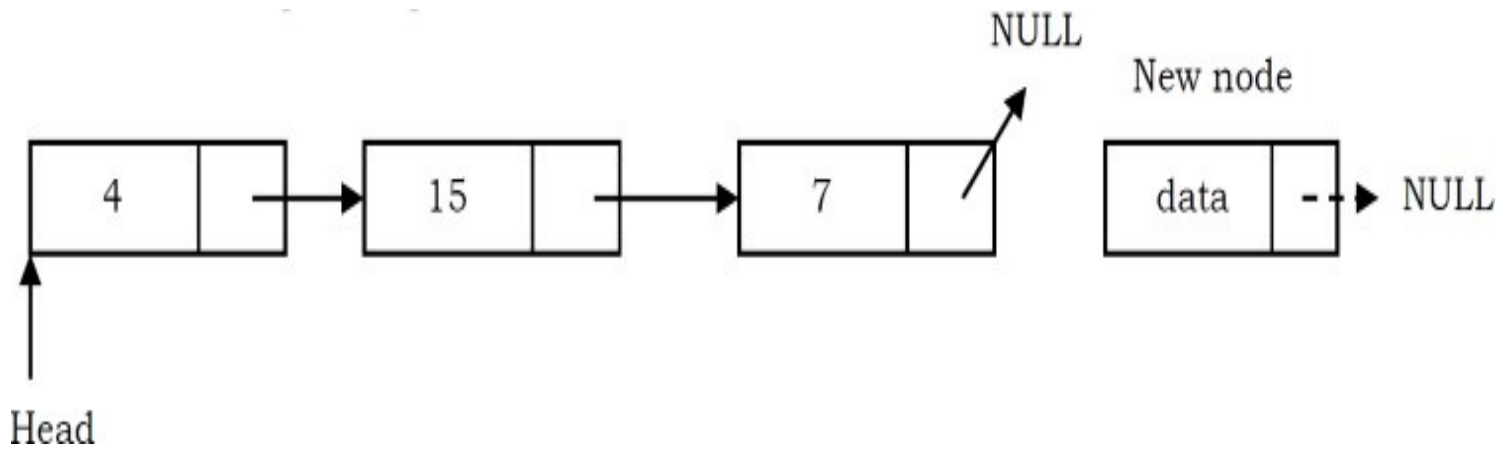
New node



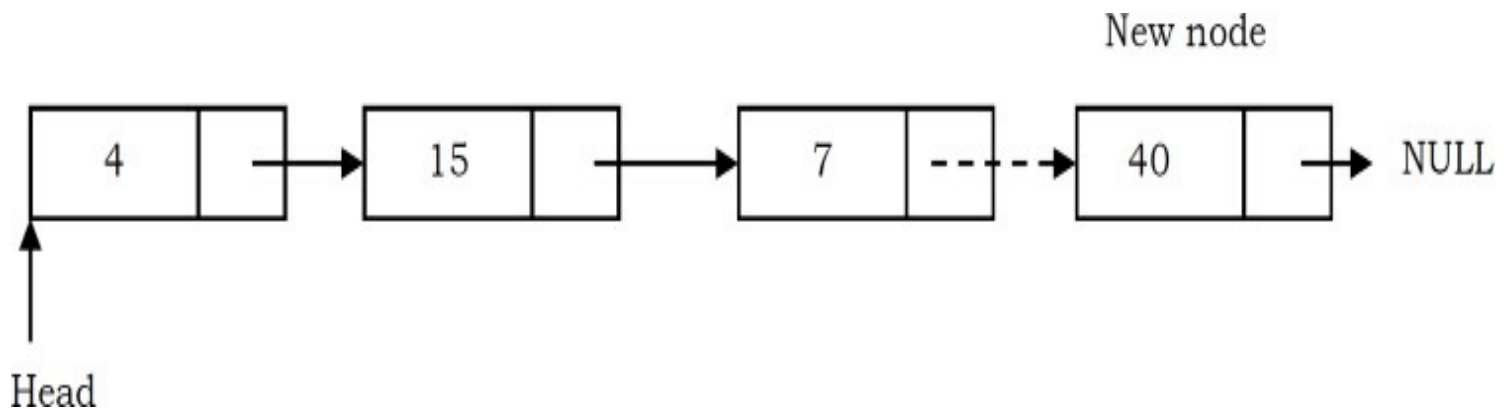
Inserting a Node in Singly Linked List at the Ending

In this case, we need to modify *two next pointers* (last nodes next pointer and new nodes next pointer).

- New nodes next pointer points to NULL.



- Last nodes next pointer points to the new node.

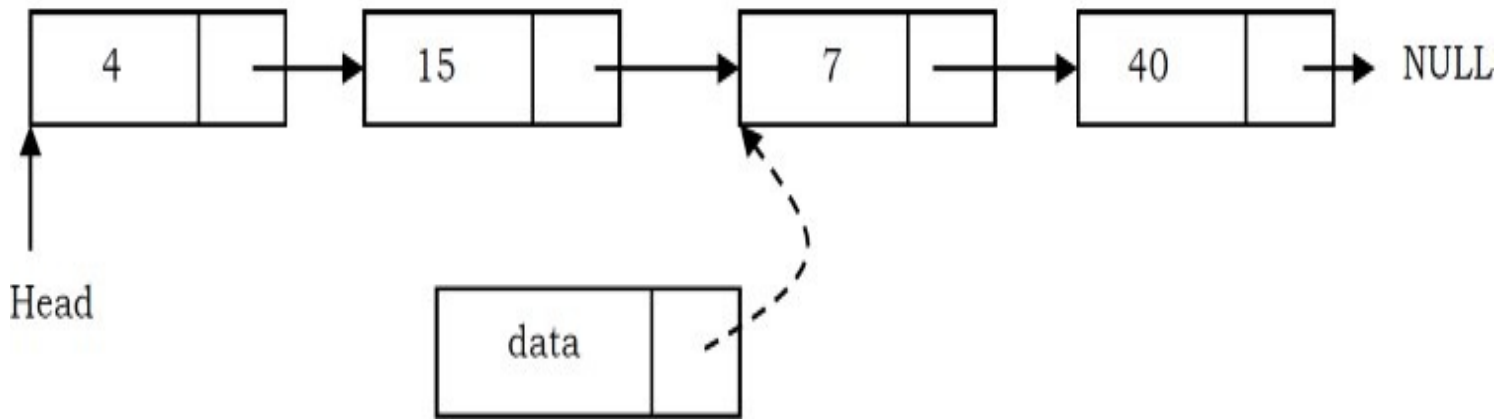


Inserting a Node in Singly Linked List at the Middle

Let us assume that we are given a position where we want to insert the new node. In this case also, we need to modify two next pointers.

- If we want to add an element at position 3 then we stop at position 2. That means we traverse 2 nodes and insert the new node. For simplicity let us assume that the second node is called *position* node. The new node points to the next node of the position where we want to add this node.

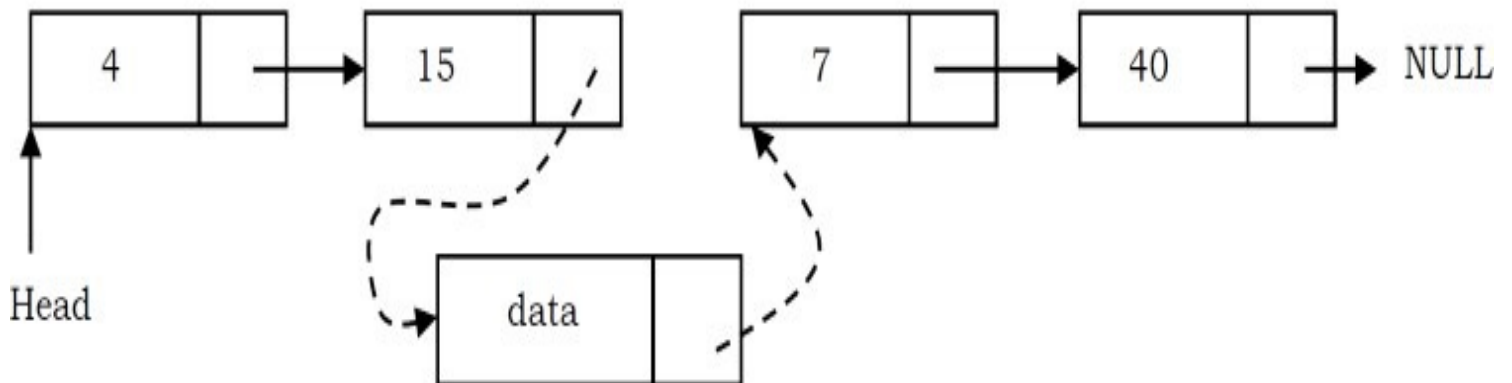
Position node



New node

- Position node's next pointer now points to the new node.

Position node



New node

Let us write the code for all three cases. We must update the first element pointer in the calling function, not just in the called function. For this reason we need to send a double pointer. The following code inserts a node in the singly linked list.

```

void InsertInLinkedList(struct ListNode **head,int data,int position) {
    int k=1;
    struct ListNode *p,*q,*newNode;
    newNode = (ListNode *)malloc(sizeof(struct ListNode));
    if(!newNode){
        printf("Memory Error");
        return;
    }
    newNode->data=data;
    p=*head;
    //Inserting at the beginning
    if(position == 1){
        newNode->next=p;
        *head=newNode;
    }
    else{
        //Traverse the list until the position where we want to insert
        while((p!=NULL) && (k<position)){
            k++;
            q=p;
            p=p->next;
        }
        q->next=newNode; //more optimum way to do this
        newNode->next=p;
    }
}

```

Note: We can implement the three variations of the *insert* operation separately.

Time Complexity: $O(n)$, since, in the worst case, we may need to insert the node at the end of the list.

Space Complexity: $O(1)$, for creating one temporary variable.

Singly Linked List Deletion

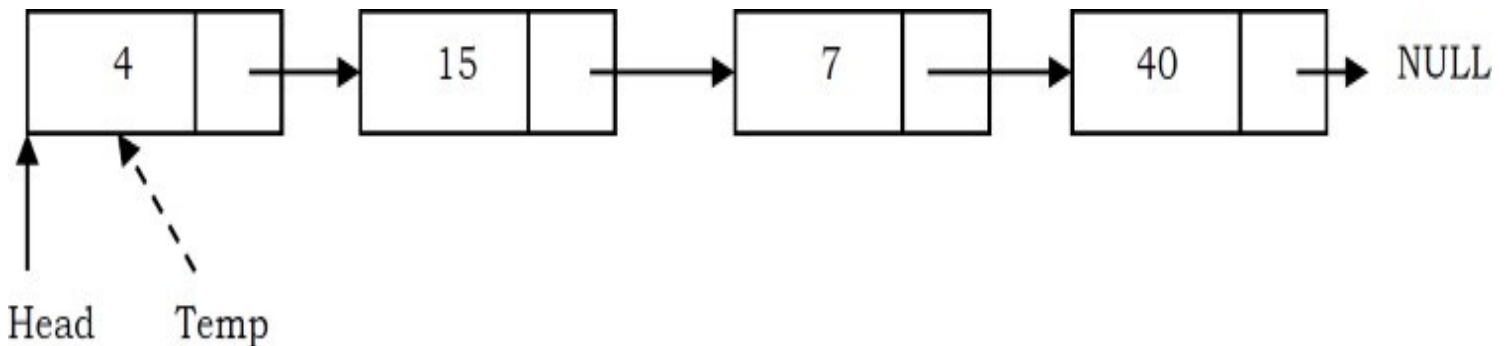
Similar to insertion, here we also have three cases.

- Deleting the first node
- Deleting the last node
- Deleting an intermediate node.

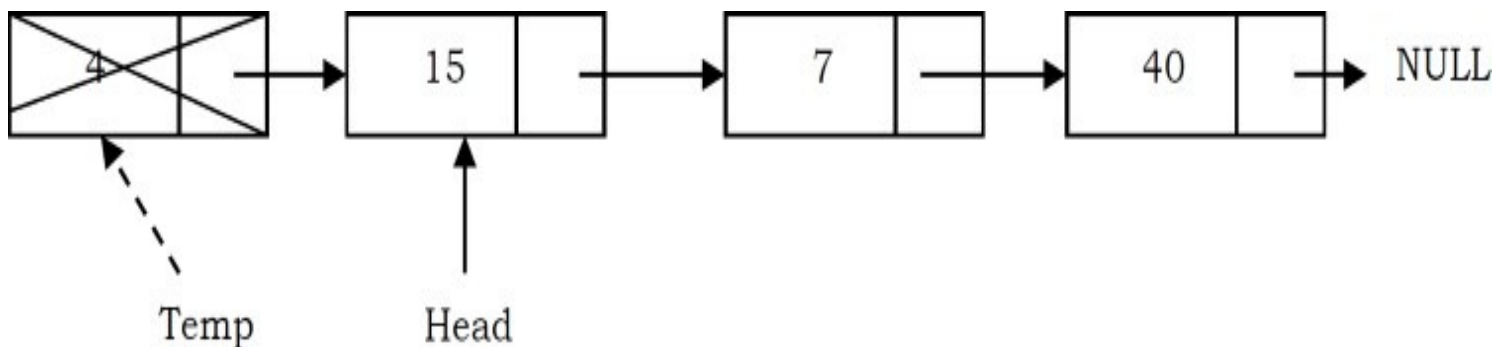
Deleting the First Node in Singly Linked List

First node (current head node) is removed from the list. It can be done in two steps:

- Create a temporary node which will point to the same node as that of head.



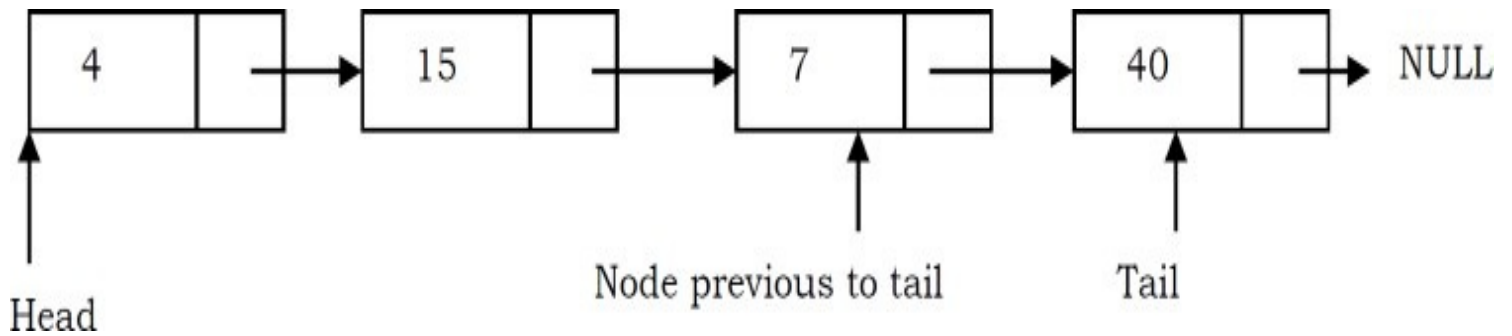
- Now, move the head nodes pointer to the next node and dispose of the temporary node.



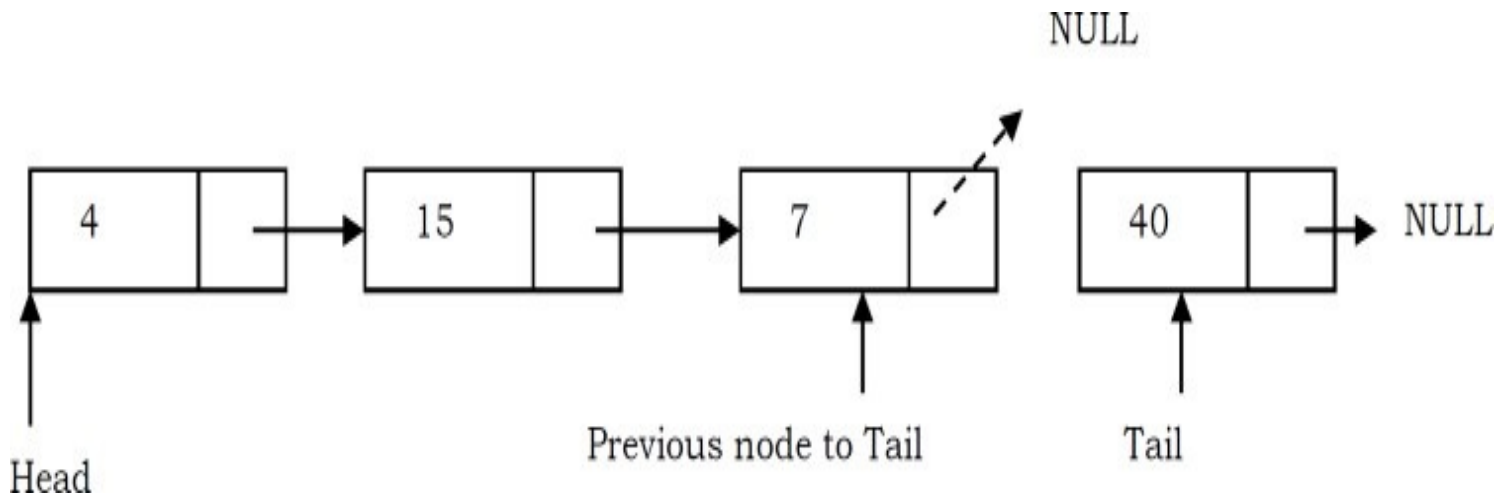
Deleting the Last Node in Singly Linked List

In this case, the last node is removed from the list. This operation is a bit trickier than removing the first node, because the algorithm should find a node, which is previous to the tail. It can be done in three steps:

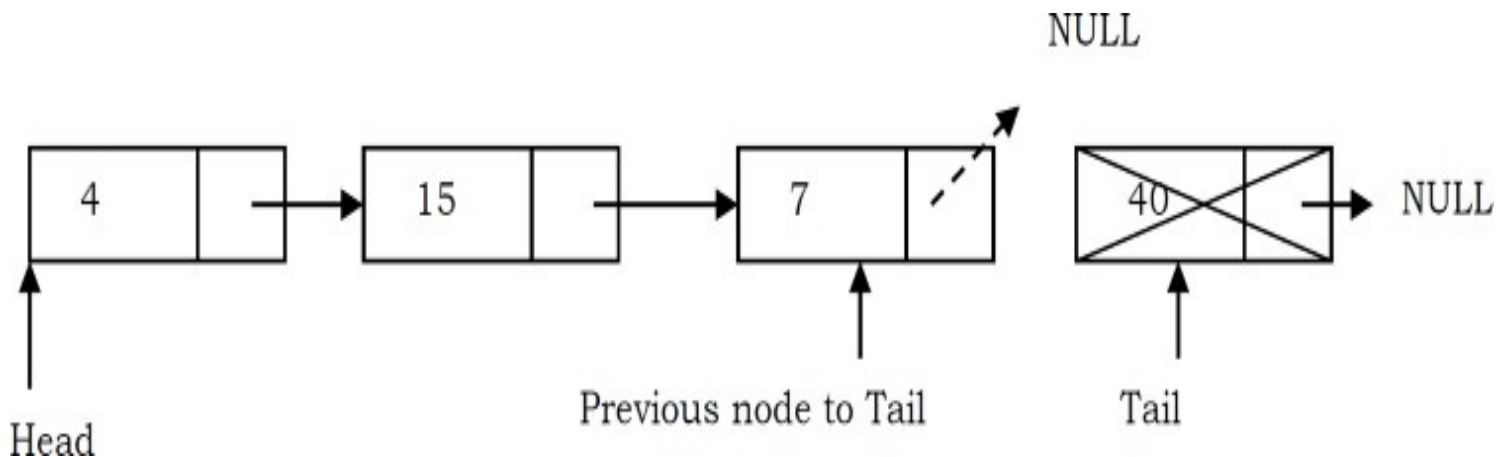
- Traverse the list and while traversing maintain the previous node address also. By the time we reach the end of the list, we will have two pointers, one pointing to the *tail* node and the other pointing to the node *before* the tail node.



- Update previous node's next pointer with NULL.



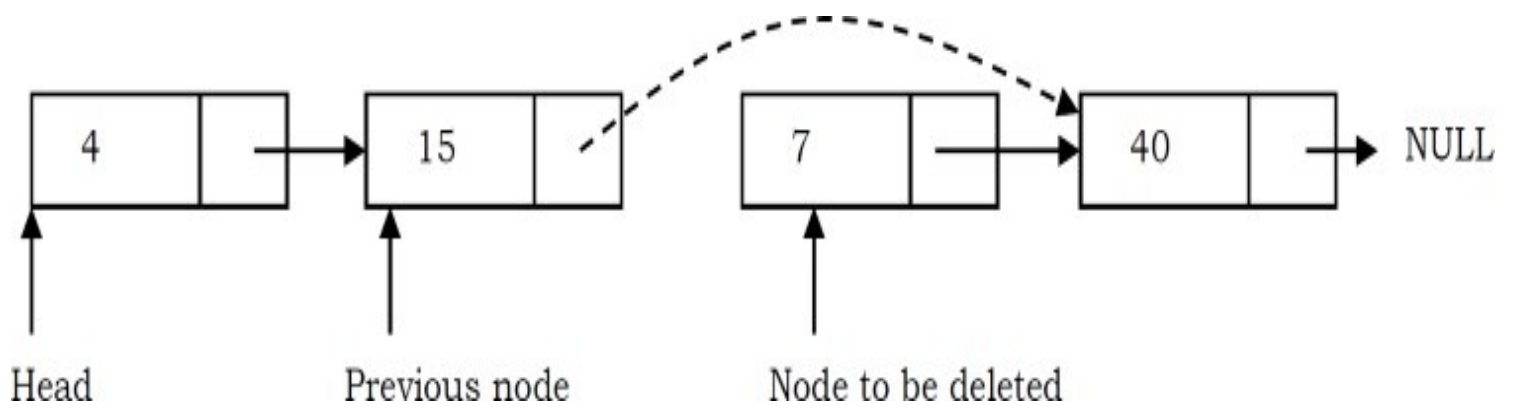
- Dispose of the tail node.



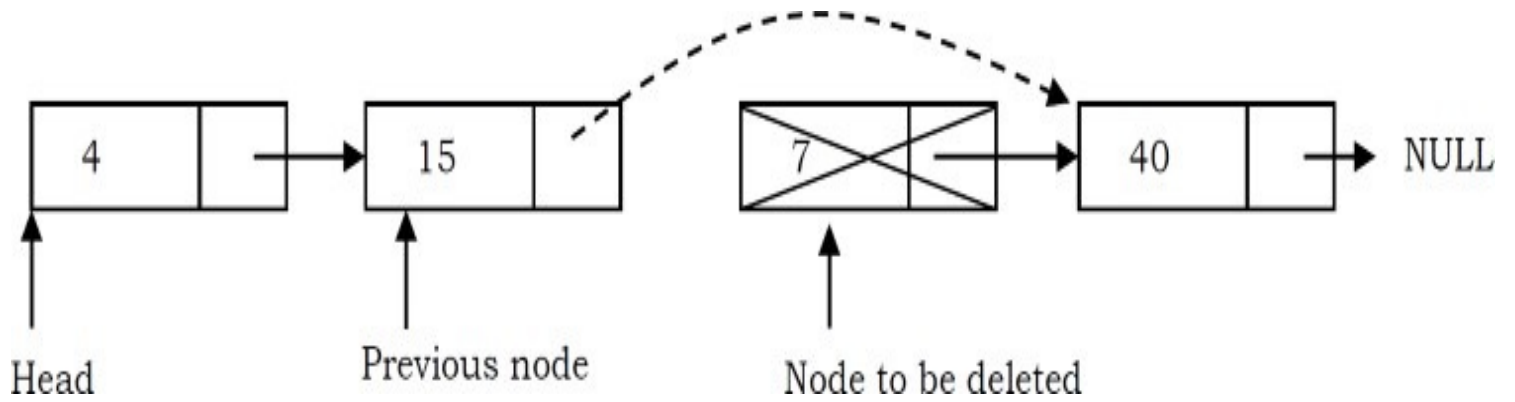
Deleting an Intermediate Node in Singly Linked List

In this case, the node to be removed is *always located between two nodes*. Head and tail links are not updated in this case. Such a removal can be done in two steps:

- Similar to the previous case, maintain the previous node while traversing the list. Once we find the node to be deleted, change the previous node's next pointer to the next pointer of the node to be deleted.



- Dispose of the current node to be deleted.



```

void DeleteNodeFromLinkedList (struct ListNode **head, int position) {
    int k = 1;
    struct ListNode *p, *q;
    if(*head == NULL) {
        printf ("List Empty");
        return;
    }
    p = *head;
    /* from the beginning */
    if(position == 1) {
        *head = (*head)→next;
        free (p);
        return;
    }
    else {
        // Traverse the list until arriving at the position from which we want to delete
        while ((p != NULL) && (k < position )) {
            k++;
            q = p;
            p = p→next;
        }
        if(p == NULL) /* At the end */
            printf ("Position does not exist.");
        else { /* From the middle */
            q→next = p→next;
            free(p);
        }
    }
}

```

Time Complexity: $O(n)$. In the worst case, we may need to delete the node at the end of the list.
 Space Complexity: $O(1)$, for one temporary variable.

Deleting Singly Linked List

This works by storing the current node in some temporary variable and freeing the current node. After freeing the current node, go to the next node with a temporary variable and repeat this process for all nodes.

```
void DeleteLinkedList(struct ListNode **head) {
    struct ListNode *auxiliaryNode, *iterator;
    iterator = *head;
    while (iterator) {
        auxiliaryNode = iterator->next;
        free(iterator);
        iterator = auxiliaryNode;
    }
    *head = NULL;           // to affect the real head back in the caller.
}
```

Time Complexity: $O(n)$, for scanning the complete list of size n .

Space Complexity: $O(1)$, for creating one temporary variable.

3.7 Doubly Linked Lists

The *advantage* of a doubly linked list (also called *two – way linked list*) is that given a node in the list, we can navigate in both directions. A node in a singly linked list cannot be removed unless we have the pointer to its predecessor. But in a doubly linked list, we can delete a node even if we don't have the previous node's address (since each node has a left pointer pointing to the previous node and can move backward).

The primary *disadvantages* of doubly linked lists are:

- Each node requires an extra pointer, requiring more space.
- The insertion or deletion of a node takes a bit longer (more pointer operations).

Similar to a singly linked list, let us implement the operations of a doubly linked list. If you understand the singly linked list operations, then doubly linked list operations are obvious. Following is a type declaration for a doubly linked list of integers:

```

struct DLLNode {
    int data;
    struct DLLNode *next;
    struct DLLNode *prev;
};

```

Doubly Linked List Insertion

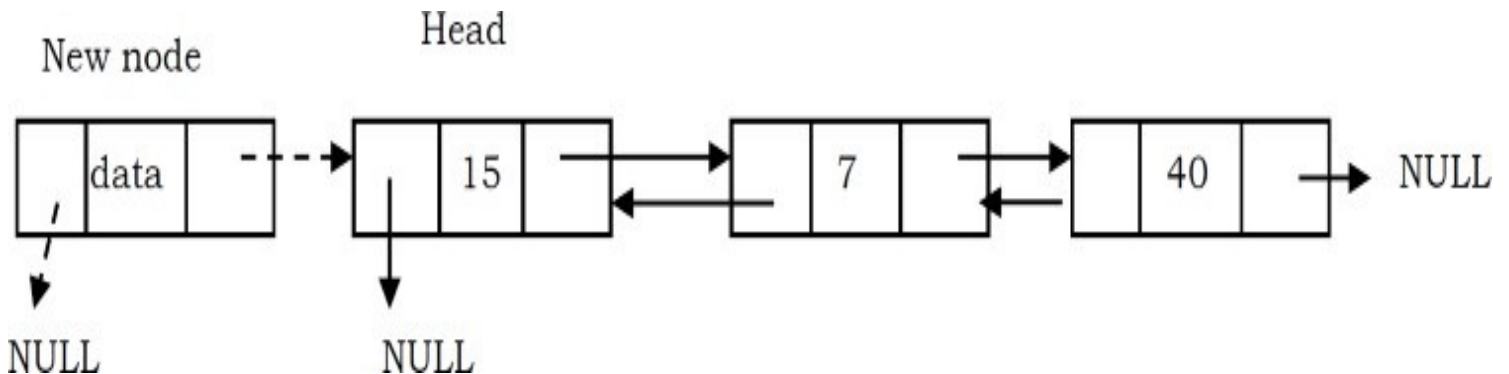
Insertion into a doubly-linked list has three cases (same as singly linked list):

- Inserting a new node before the head.
- Inserting a new node after the tail (at the end of the list).
- Inserting a new node at the middle of the list.

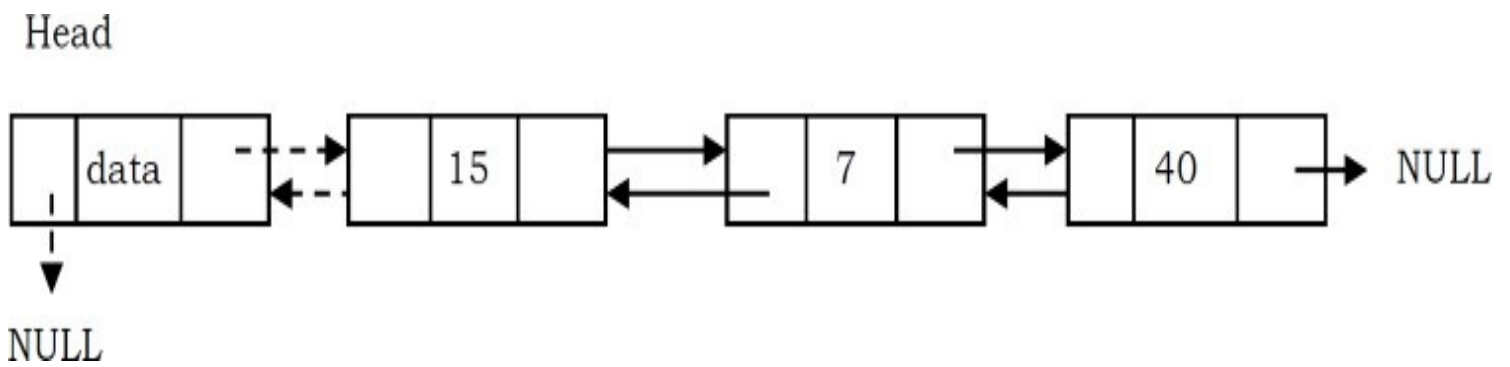
Inserting a Node in Doubly Linked List at the Beginning

In this case, new node is inserted before the head node. Previous and next pointers need to be modified and it can be done in two steps:

- Update the right pointer of the new node to point to the current head node (dotted link in below figure) and also make left pointer of new node as NULL.



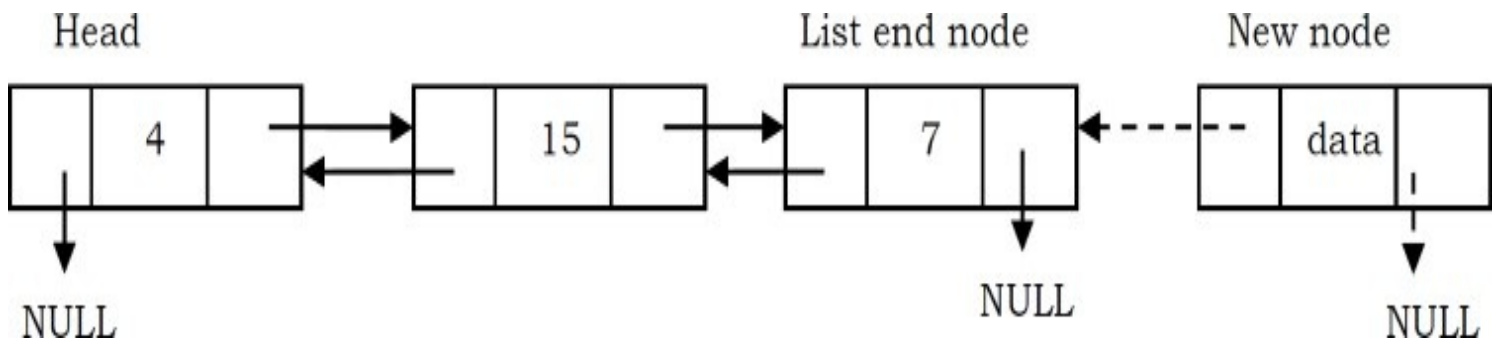
- Update head node's left pointer to point to the new node and make new node as head. Head



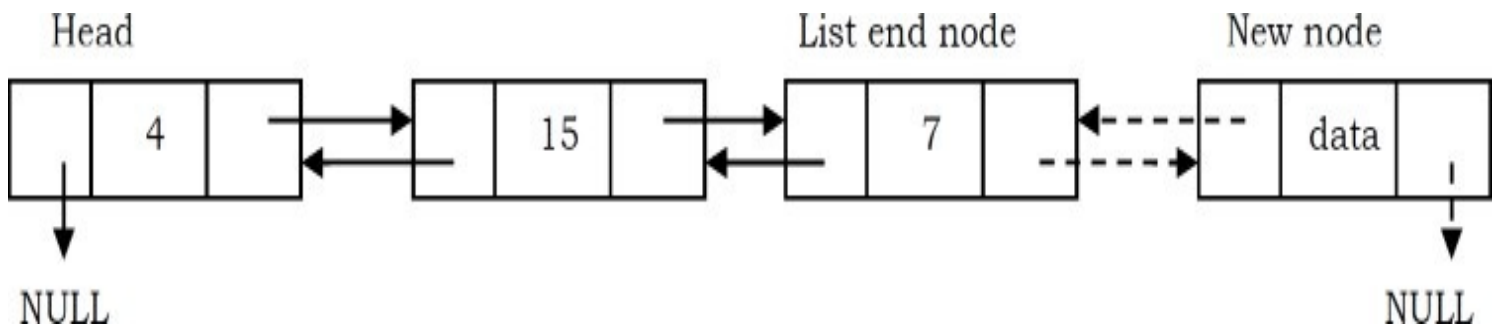
Inserting a Node in Doubly Linked List at the Ending

In this case, traverse the list till the end and insert the new node.

- New node right pointer points to NULL and left pointer points to the end of the list.



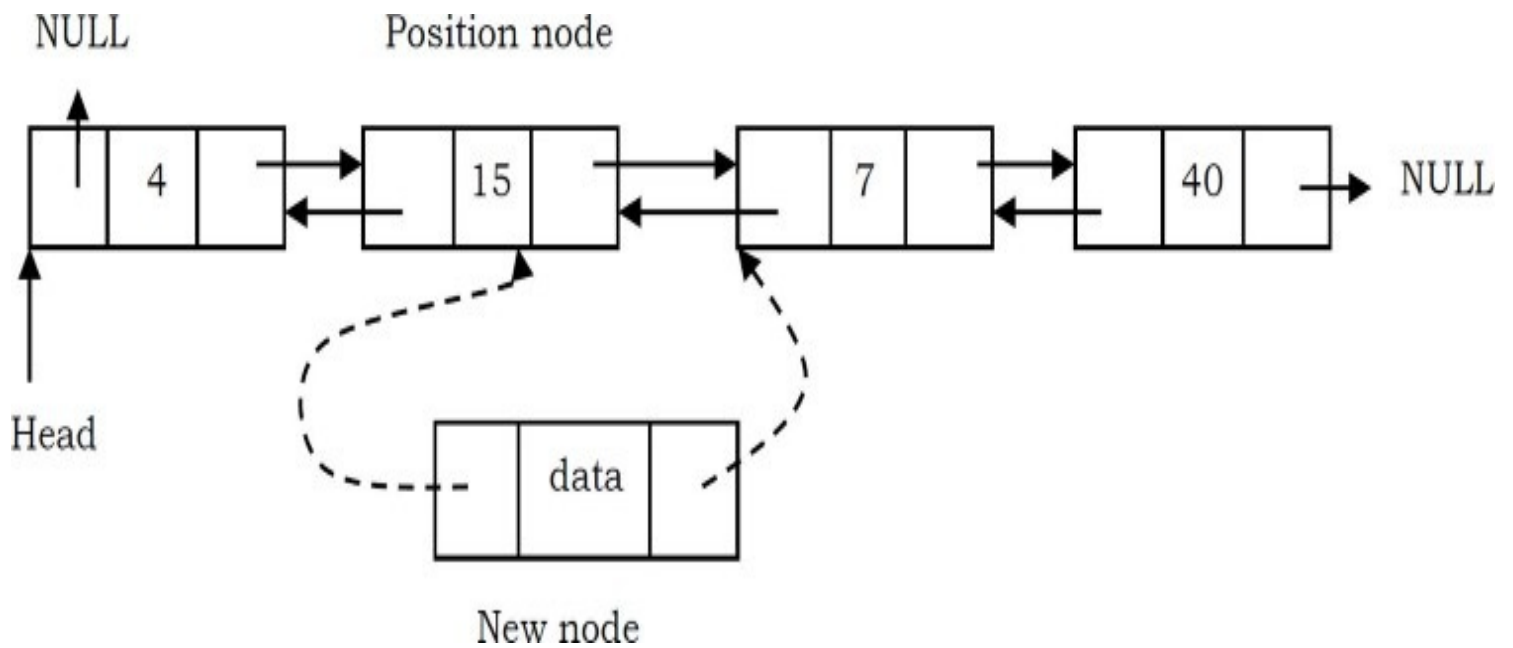
- Update right pointer of last node to point to new node.



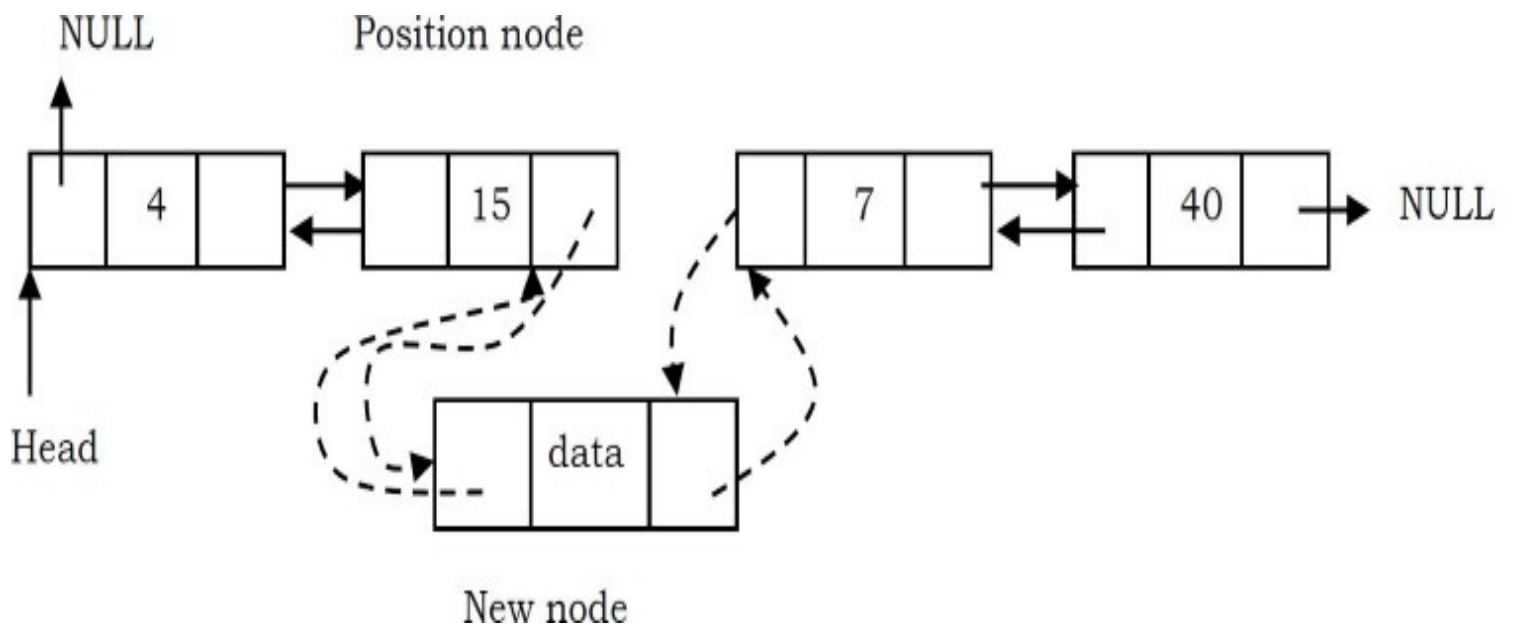
Inserting a Node in Doubly Linked List at the Middle

As discussed in singly linked lists, traverse the list to the position node and insert the new node.

- New node* right pointer points to the next node of the *position node* where we want to insert the new node. Also, *new node* left pointer points to the *position node*.



- Position node right pointer points to the new node and the *next node* of position node left pointer points to new node.



Now, let us write the code for all of these three cases. We must update the first element pointer in the calling function, not just in the called function. For this reason we need to send a double pointer. The following code inserts a node in the doubly linked list

```

void DLLInsert(struct DLLNode **head, int data, int position) {
    int k = 1;
    struct DLLNode *temp, *newNode;
    newNode = (struct DLLNode *) malloc(sizeof ( struct DLLNode ));
    if(!newNode) {                                     //Always check for memory errors
        printf ("Memory Error");
        return;
    }
    newNode->data = data;
    if(position == 1) {                                //Inserting a node at the beginning
        newNode->next = *head;
        newNode->prev = NULL;

        if(*head)
            (*head)->prev = newNode;

        *head = newNode;
        return;
    }
    temp = *head;
    while ( (k < position - 1) && temp->next!=NULL) {
        temp = temp->next;
        k++;
    }
    if(k!=position){
        printf("Desired position does not exist\n");
    }
    newNode->next=temp->next;
    newNode->prev=temp;
    if(temp->next)
        temp->next->prev=newNode;

    temp->next=newNode;
    return;
}

```

Time Complexity: $O(n)$. In the worst case, we may need to insert the node at the end of the list.
Space Complexity: $O(1)$, for creating one temporary variable.

Doubly Linked List Deletion

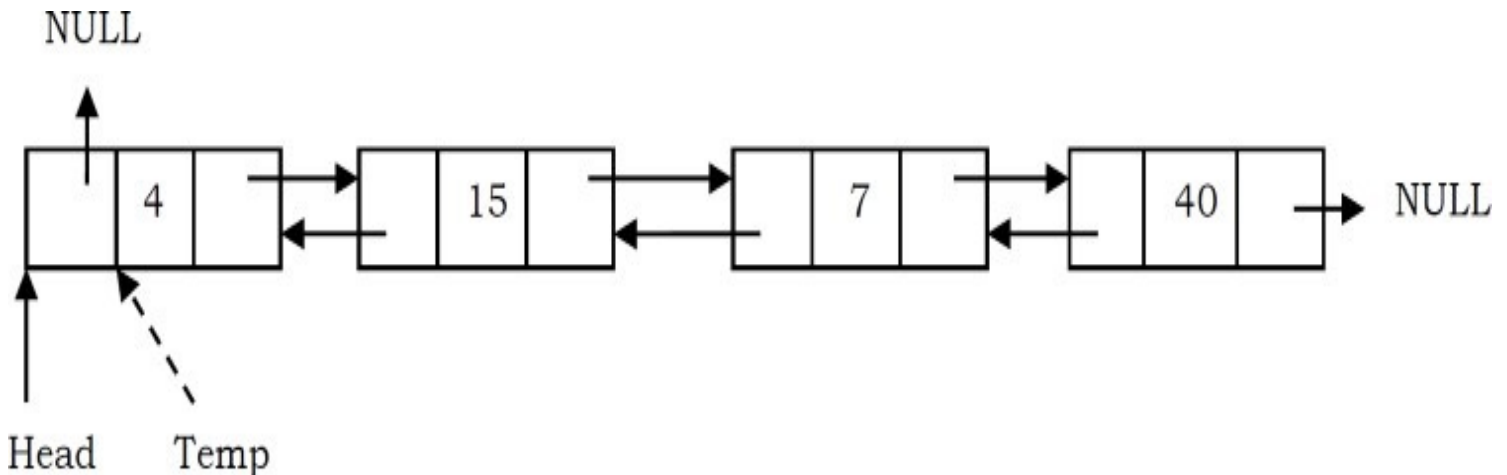
Similar to singly linked list deletion, here we have three cases:

- Deleting the first node
- Deleting the last node
- Deleting an intermediate node

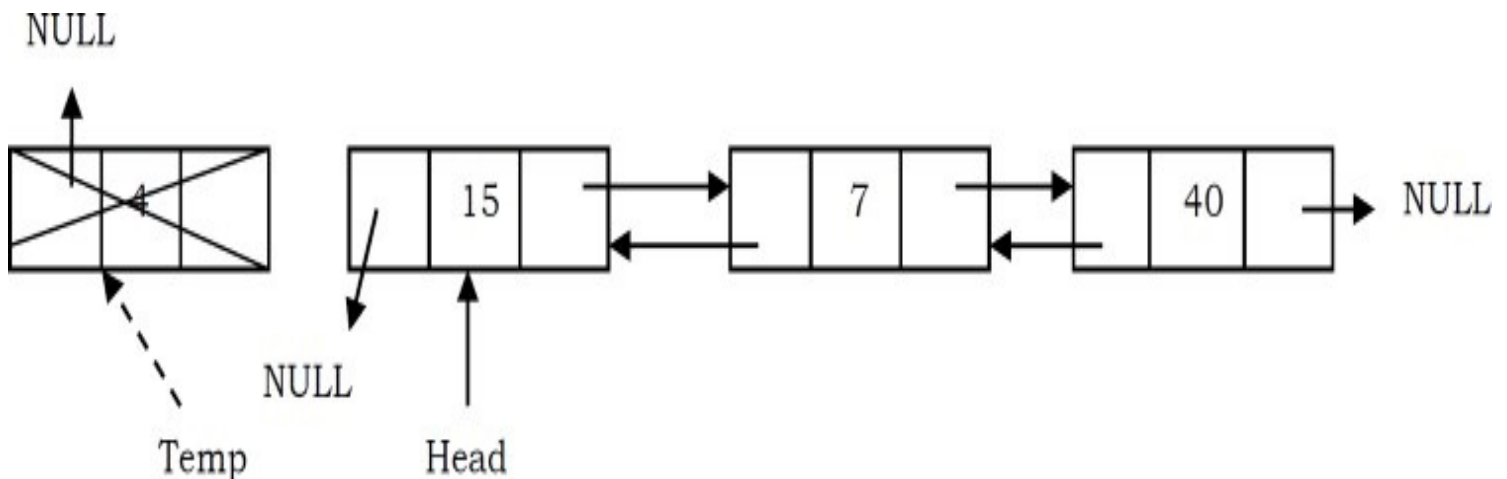
Deleting the First Node in Doubly Linked List

In this case, the first node (current head node) is removed from the list. It can be done in two steps:

- Create a temporary node which will point to the same node as that of head.



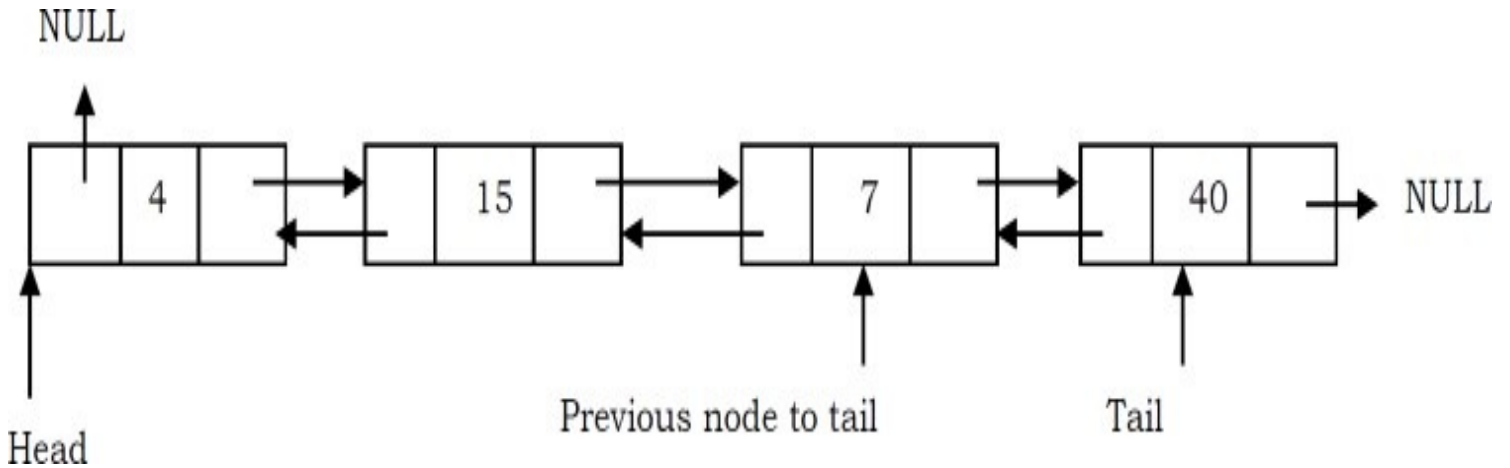
- Now, move the head nodes pointer to the next node and change the heads left pointer to NULL. Then, dispose of the temporary node.



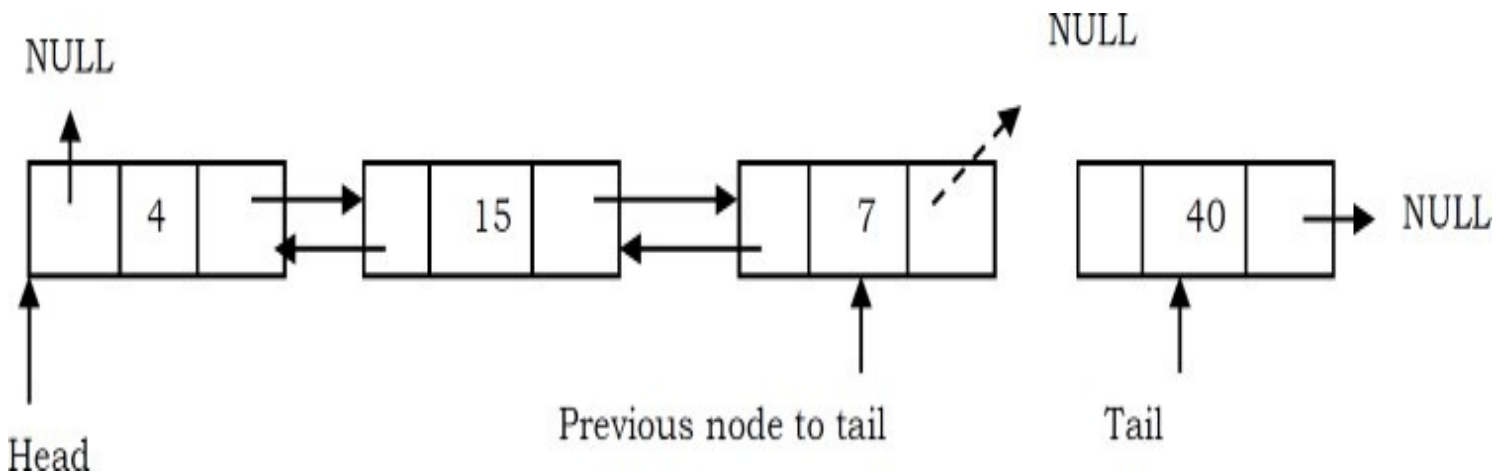
Deleting the Last Node in Doubly Linked List

This operation is a bit trickier than removing the first node, because the algorithm should find a node, which is previous to the tail first. This can be done in three steps:

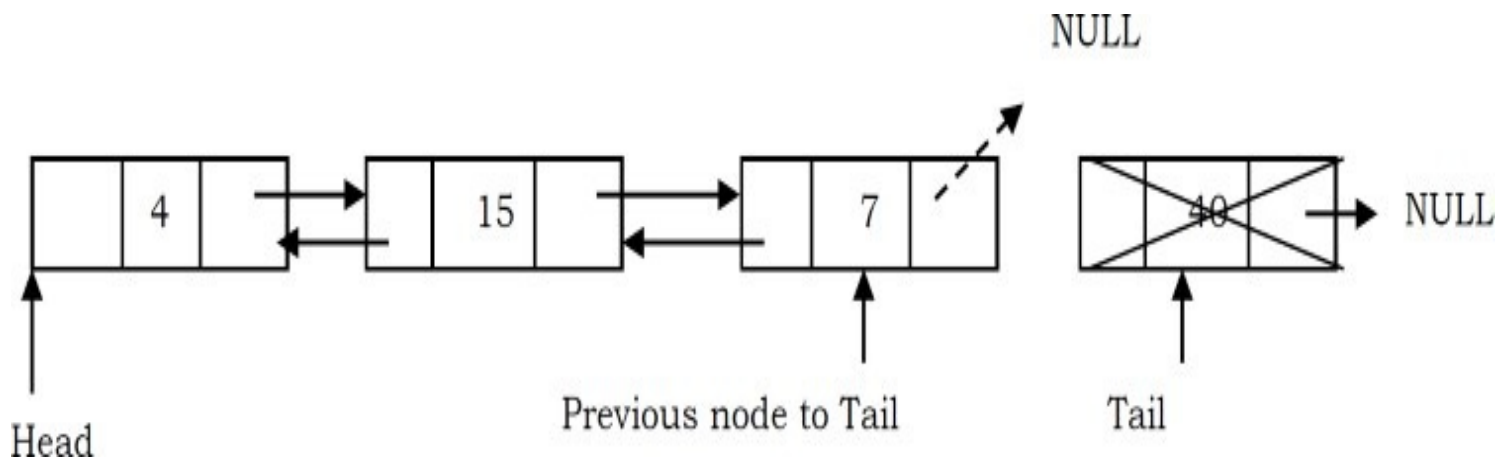
- Traverse the list and while traversing maintain the previous node address also. By the time we reach the end of the list, we will have two pointers, one pointing to the tail and the other pointing to the node before the tail.



- Update the next pointer of previous node to the tail node with NULL.



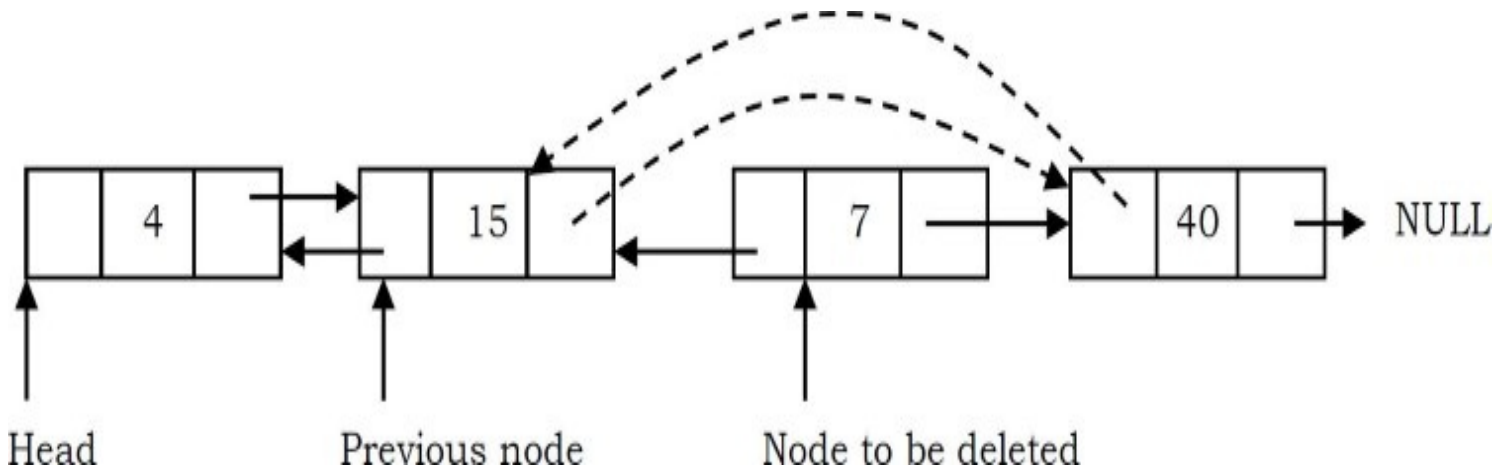
- Dispose the tail node.



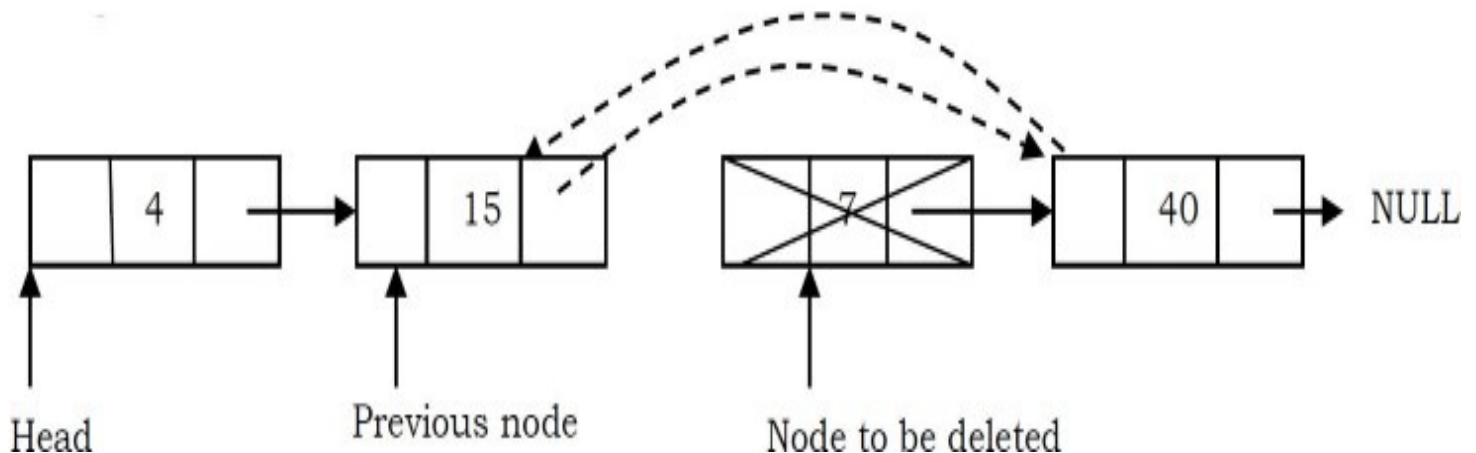
Deleting an Intermediate Node in Doubly Linked List

In this case, the node to be removed is *always located between* two nodes, and the head and tail links are not updated. The removal can be done in two steps:

- Similar to the previous case, maintain the previous node while also traversing the list. Upon locating the node to be deleted, change the previous node's next pointer to the next node of the node to be deleted.



- Dispose of the current node to be deleted.



```

void DLLDelete(struct DLLNode **head, int position) {
    struct DLLNode *temp, *temp2, temp = *head;
    int k = 1;
    if(*head == NULL) {
        printf("List is empty");
        return;
    }
    if(position == 1) {
        *head = (*head)→next;

        if(*head != NULL)
            (*head)→prev = NULL;
        free(temp);
        return;
    }
    while((k < position) && temp→next!=NULL) {
        temp = temp→next;
        k++;
    }
    if(k!=position-1){
        printf("Desired position does not exist\n");
    }

    temp2=temp→prev;
    temp2→next=temp→next;

    if(temp→next) // Deletion from Intermediate Node
        temp→next→prev=temp2;
    free(temp);
    return;
}

```

Time Complexity: $O(n)$, for scanning the complete list of size n .

Space Complexity: $O(1)$, for creating one temporary variable.

3.8 Circular Linked Lists

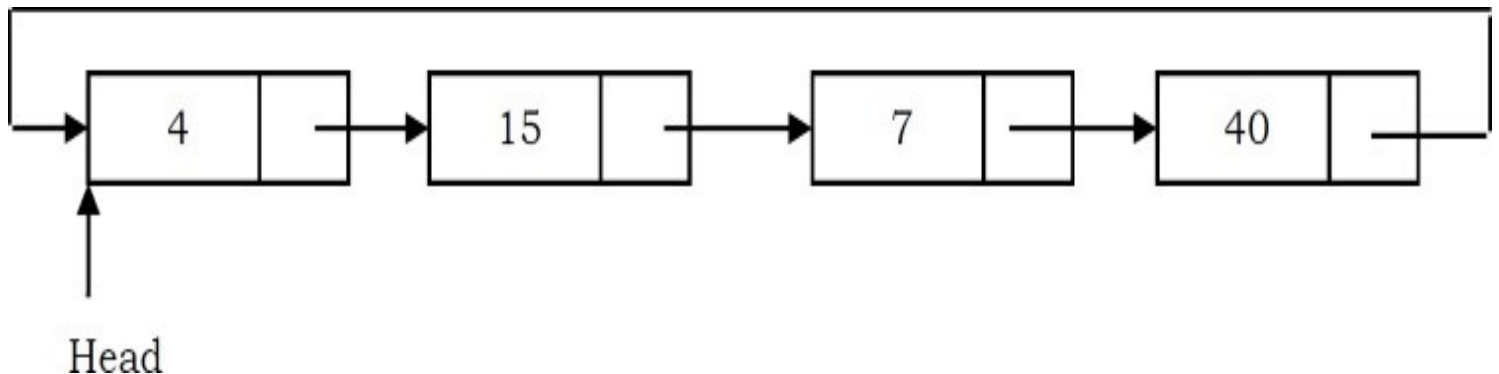
In singly linked lists and doubly linked lists, the end of lists are indicated with NULL value. But circular linked lists do not have ends. While traversing the circular linked lists we should be careful; otherwise we will be traversing the list infinitely. In circular linked lists, each node has a successor. Note that unlike singly linked lists, there is no node with NULL pointer in a circularly linked list. In some situations, circular linked lists are useful.

For example, when several processes are using the same computer resource (CPU) for the same amount of time, we have to assure that no process accesses the resource before all other processes do (round robin algorithm). The following is a type declaration for a circular linked list of integers:

```
typedef struct CLLNode {  
    int data;  
    struct ListNode *next;  
};
```

In a circular linked list, we access the elements using the *head* node (similar to *head* node in singly linked list and doubly linked lists).

Counting Nodes in a Circular Linked List



The circular list is accessible through the node marked *head*. To count the nodes, the list has to be traversed from the node marked *head*, with the help of a dummy node *current*, and stop the counting when *current* reaches the starting node *head*.

If the list is empty, *head* will be NULL, and in that case set *count* = 0. Otherwise, set the current pointer to the first node, and keep on counting till the current pointer reaches the starting node.


```

int CircularListLength(struct CLLNode *head) {
    struct CLLNode *current = head;
    int count = 0;
    if(head == NULL)
        return 0;

    do {
        current = current→next;
        count++;
    } while (current != head);

    return count;
}

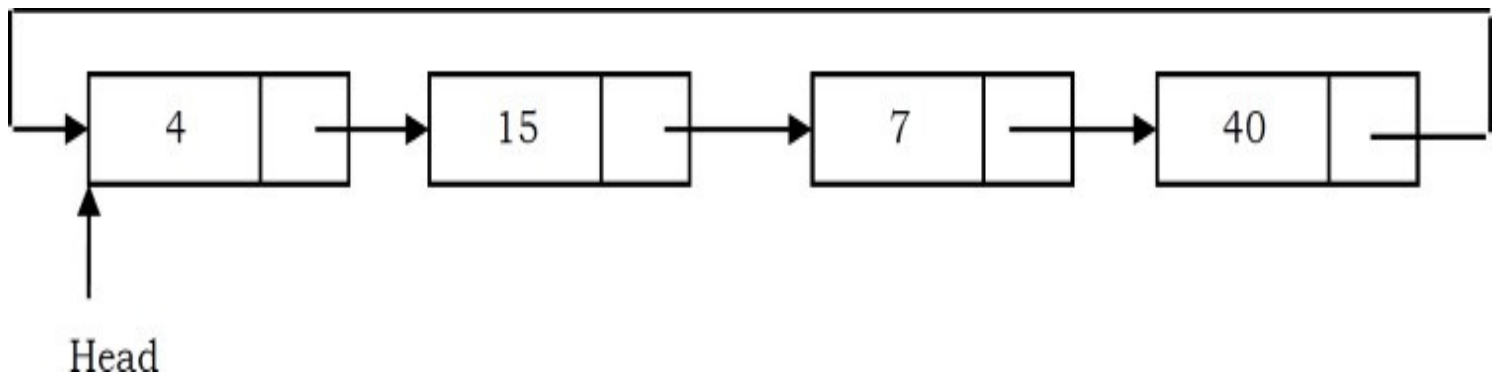
```

Time Complexity: $O(n)$, for scanning the complete list of size n .

Space Complexity: $O(1)$, for creating one temporary variable.

Printing the Contents of a Circular Linked List

We assume here that the list is being accessed by its *head* node. Since all the nodes are arranged in a circular fashion, the *tail* node of the list will be the node previous to the *head* node. Let us assume we want to print the contents of the nodes starting with the *head* node. Print its contents, move to the next node and continue printing till we reach the *head* node again.



```

void PrintCircularListData(struct CLLNode *head) {
    struct CLLNode *current = head;
    if(head == NULL)
        return;

    do {
        printf ("%d", current->data);
        current = current->next;
    } while (current != head);
}

```

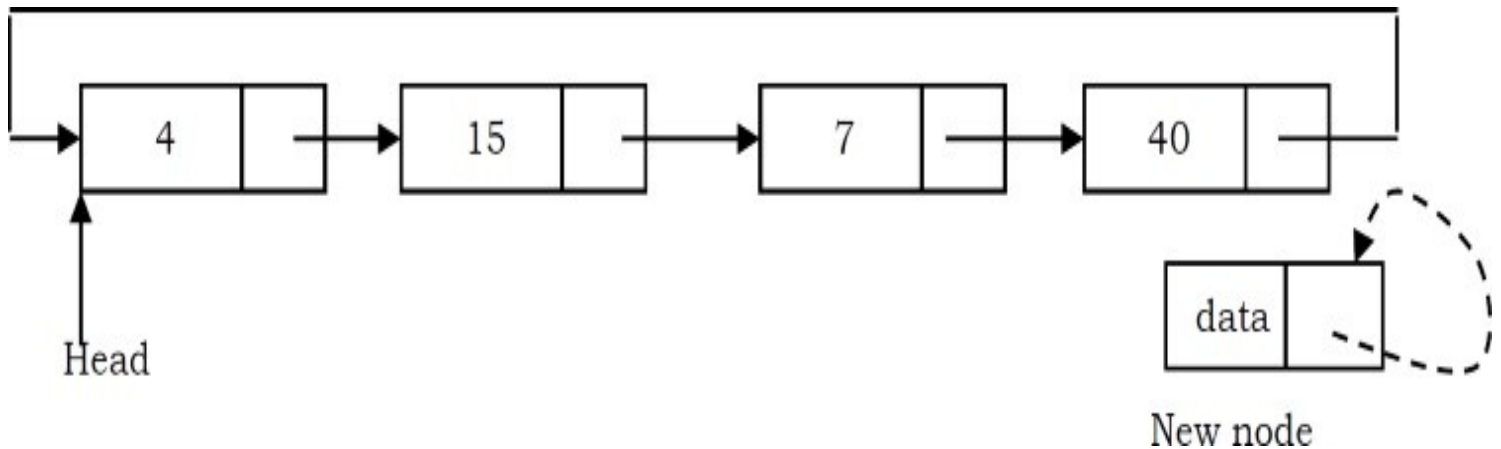
Time Complexity: $O(n)$, for scanning the complete list of size n .

Space Complexity: $O(1)$, for temporary variable.

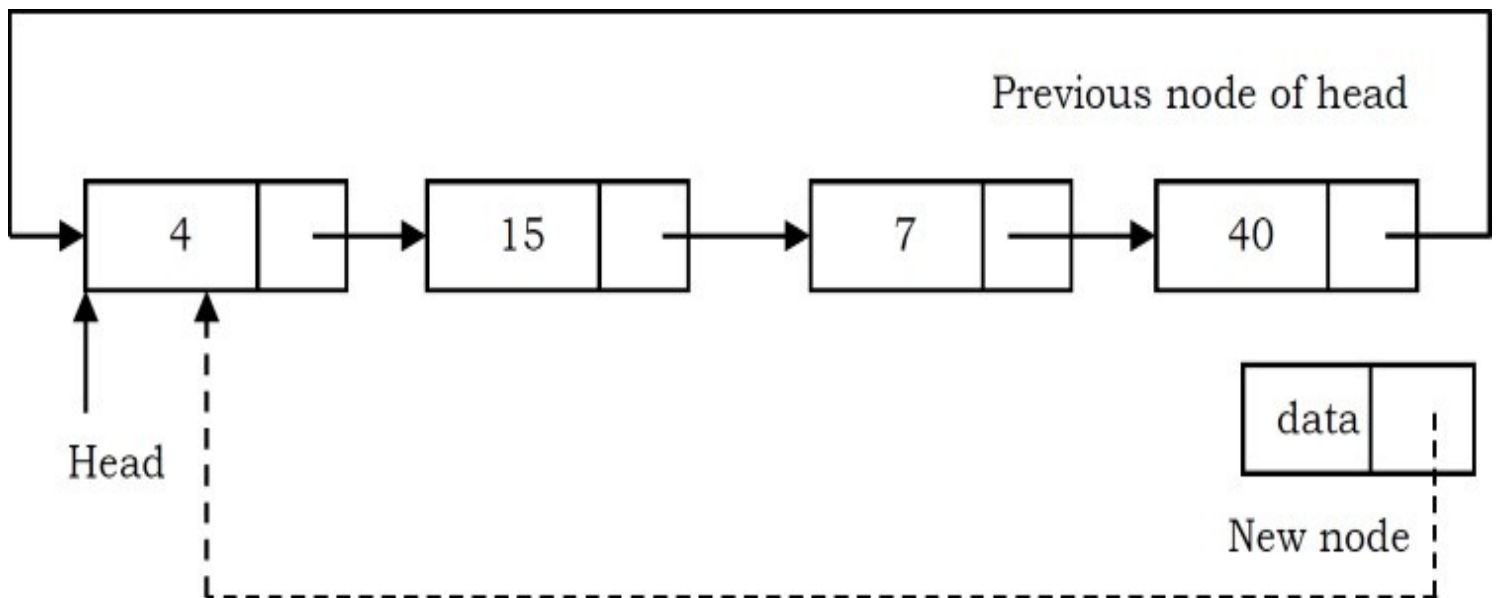
Inserting a Node at the End of a Circular Linked List

Let us add a node containing *data*, at the end of a list (circular list) headed by *head*. The new node will be placed just after the tail node (which is the last node of the list), which means it will have to be inserted in between the tail node and the first node.

- Create a new node and initially keep its next pointer pointing to itself.



- Update the next pointer of the new node with the head node and also traverse the list to the tail. That means in a circular list we should stop at the node whose next node is head.



- Update the next pointer of the previous node to point to the new node and we get the list as shown below.

